

OSNOVE UMETNE INTELIGENCE

2025/26

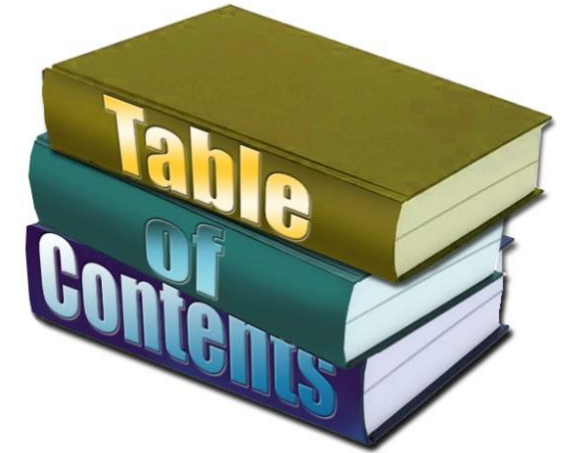
preiskovanje prostora stanj

*neinformirani preiskovalni algoritmi
(BFS, DFS, IDS, BS, UCS)*

*informirani preiskovalni algoritmi
(GS, A*, IDA*)*

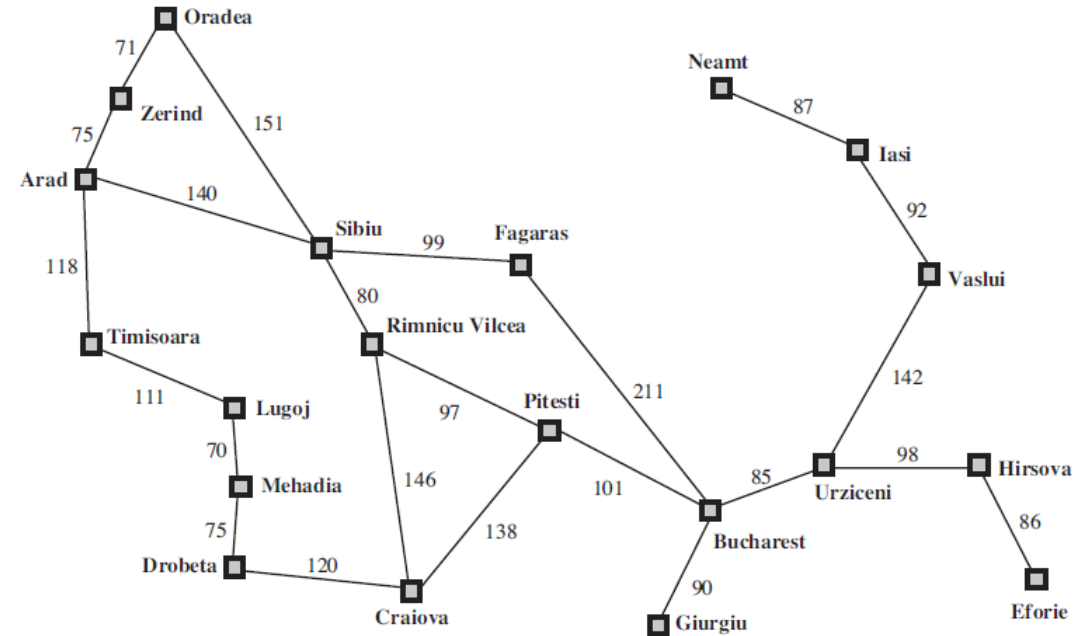
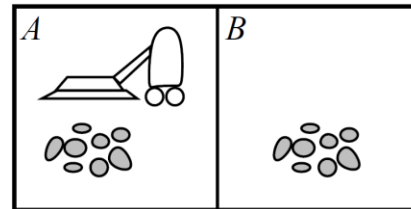
Pregled

- preiskovanje prostora stanj
 - definicija in cilji področja
 - primeri umetnih in realnih problemov
 - neinformirani preiskovalni algoritmi
 - iskanje v širino
 - iskanje v globino
 - iterativno poglobljanje
 - dvosmerno iskanje
 - cenovno – optimalno iskanje
 - informirani preiskovalni algoritmi
 - hevristično preiskovanje
 - požrešno preiskovanje
 - A*
 - IDA*
 - kakovost hevrističnih funkcij

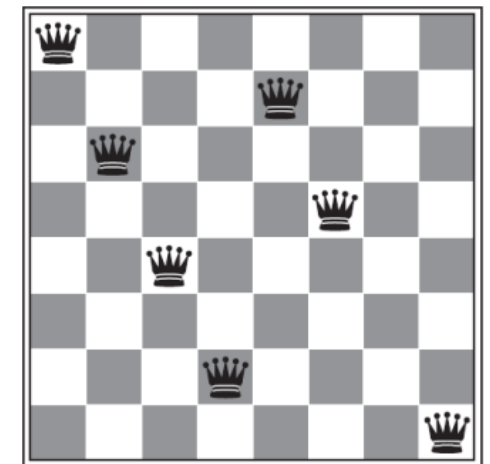
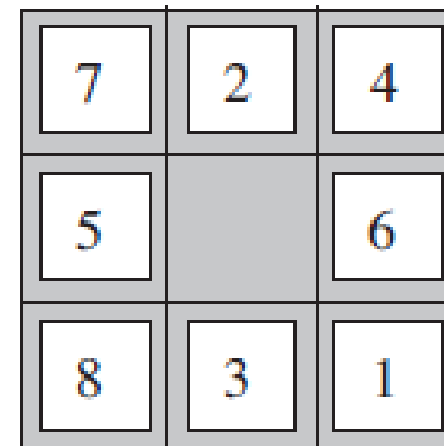


Primeri problemov

- avtomatski sesalec
- iskanje najkrajše poti
- igra 8 ploščic
- 8 kraljic na šahovnici
- planiranje v svetu kock
- manevriranje robotov
- trgovski potnik



- potrebujemo:
 1. način za abstrakcijo problema
 2. simbolično predstavitev (graf, drevo)
 3. uporabo algoritma na simbolični predstavitvi za iskanje poti do cilja



Definicija problema

Za formalni opis problema potrebujemo:

- **začetno stanje**
- opis vseh možnih **akcij**, ki so na razpolago v posameznih stanjih
- **prehodno funkcijo**, ki definira naslednika stanja
$$\text{rezultat}(\text{trenutno_stanje}, \text{akcija}) = \text{novo_stanje}$$
- **ciljni predikat**:
$$\text{cilj}(\text{stanje}) = \{\text{true}, \text{false}\}$$
- **cenilna funkcija**, ki vsaki poti določi ceno (numerično vrednost)
$$c(\text{stanje}, \text{akcija}, \text{stanje}') \geq 0$$

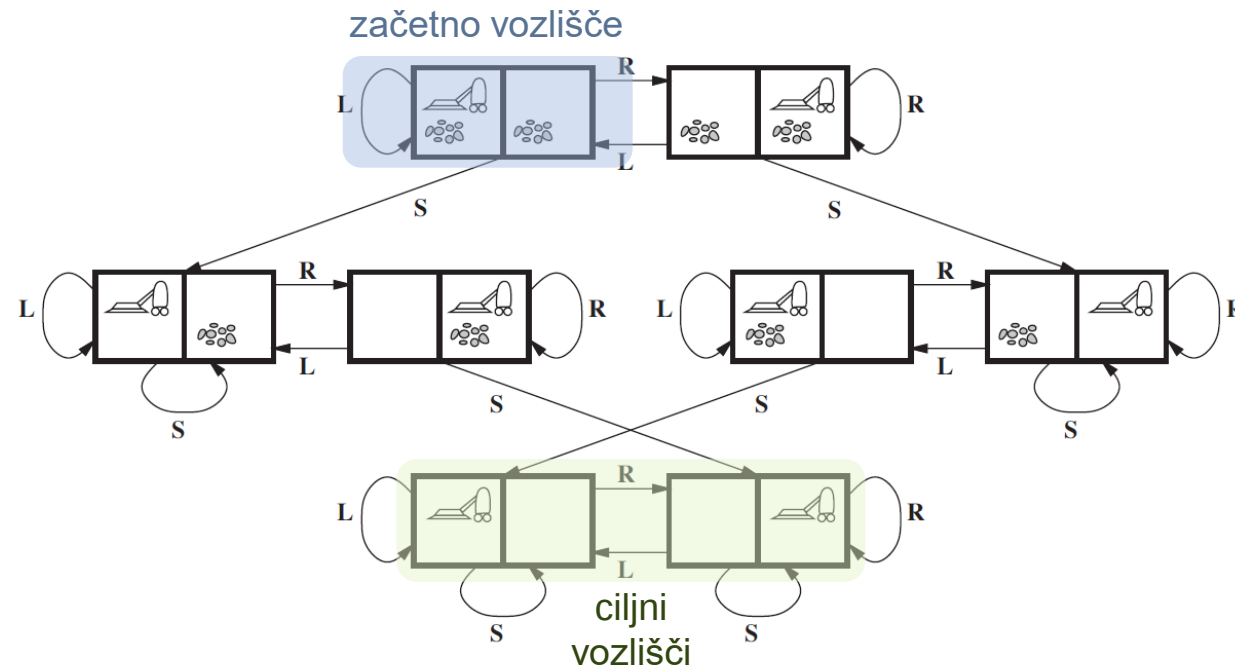


Definicija problema

- opisani način podajanja problema omogoča, da problem predstavimo z usmerjenim grafom, ki ponazarja **prostor stanj**:
 - **vozlišča**: stanja (problemske situacije)
 - **povezave**: akcije (dovoljene poteze)
 - **pot** v grafu: zaporedje stanj, ki ga povezuje zaporedje akcij
 - eno **začetno** vozlišče
 - eno ali več **ciljnih** vozlišč
- reševanje problema – **iskanje poti v grafu s preiskovanjem**
- **rešitev problema** – zaporedje akcij (pot), ki vodi od začetnega do ciljnega vozlišča
 - optimalna rešitev problema izmed vseh možnih ima najnižjo ceno poti

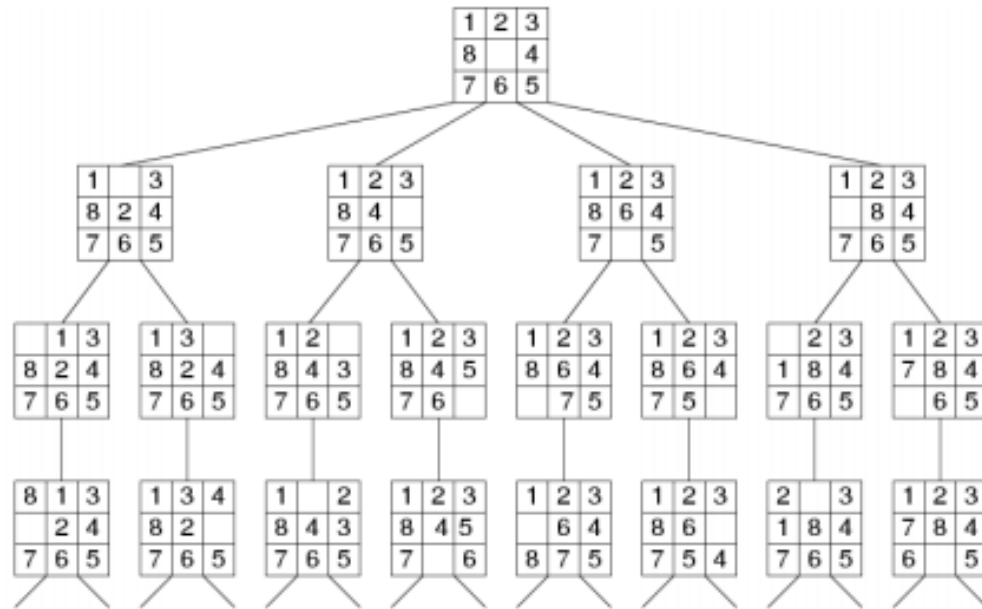


Primer: sesalec



- **stanja** – kombinacija lokacije robota in umazanije v prostorih A in B ($2 \times 2^2 = 8$ stanj)
- **akcije** – levo, desno, sesaj (in počakaj?)
- **cilj** – oba prostora čista
- **cena akcije** – npr. 1 za vsako potezo

Primer: igra 8 ploščic



7	2	4
5		6
8	3	1

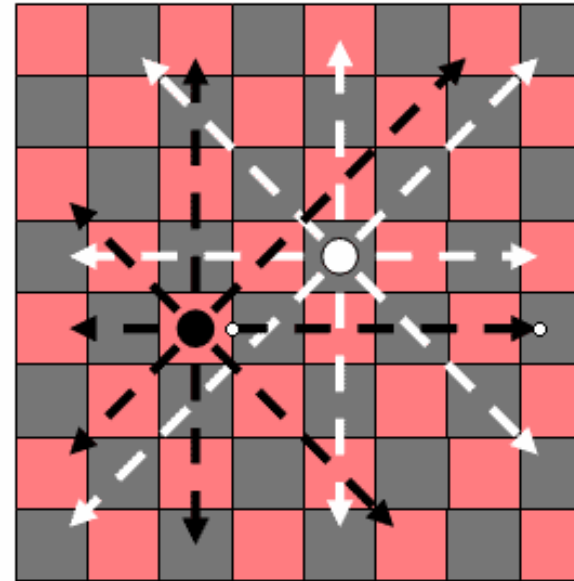
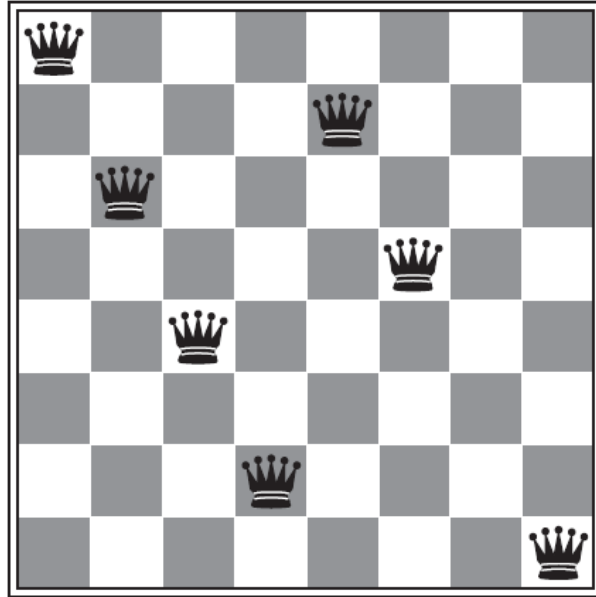
začetno stanje

	1	2
3	4	5
6	7	8

končno stanje

- **stanja** – lokacije ploščic
 - **akcije** – premiki "prazne" ploščice levo, desno, gor, dol
 - **cilj** – podano ciljno stanje
 - **cena akcije** – 1 za vsako potezo
-
- problem je NP-poln
 - igra 3x3 ima $9!/2 = 181.440$ dosegljivih stanj
 - igra 4x4 ima okoli $1,3 \times 10^{12}$ dosegljivih stanj
 - igra 5x5 ima okoli 10^{25} dosegljivih stanj

Primer: problem 8 kraljic



- **stanja** – postavitve 0-8 kraljic na plošči ($64 \times 63 \times \dots \times 57 = 1.8 \times 10^{14}$) stanj
- **začetno stanje** – prazna igralna plošča
- **akcije** – postavitve nove kraljice na prazno polje
- **cilj** – 8 kraljic na plošči, nobeni dve se ne napadata

bolj optimalna formulacija problema:

- stanja – postavitve 0-8 kraljic od leve proti desni, v vsak stolpec ena (2057 stanj)
- akcije – postavitve nove kraljice na prazno polje v prvi skrajno levi prosti stolpec

Primer: Knuthov neskončni prostor

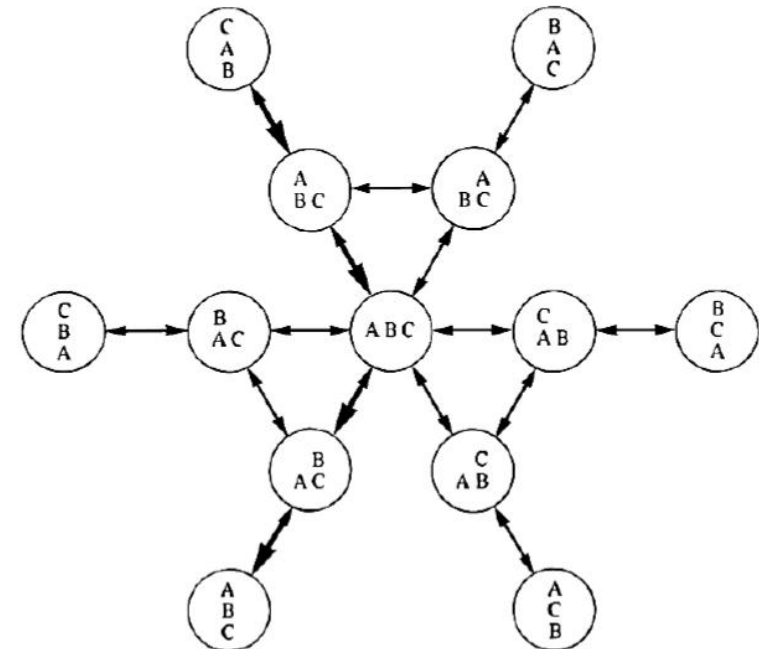
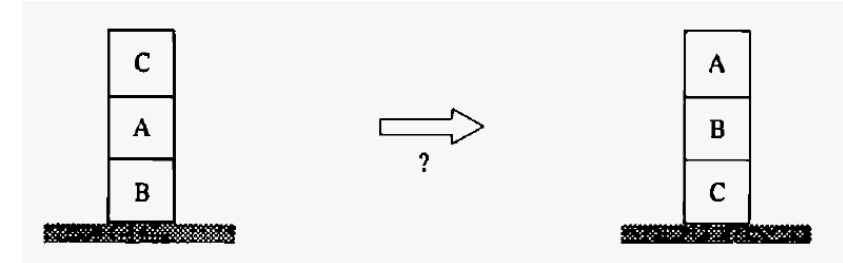
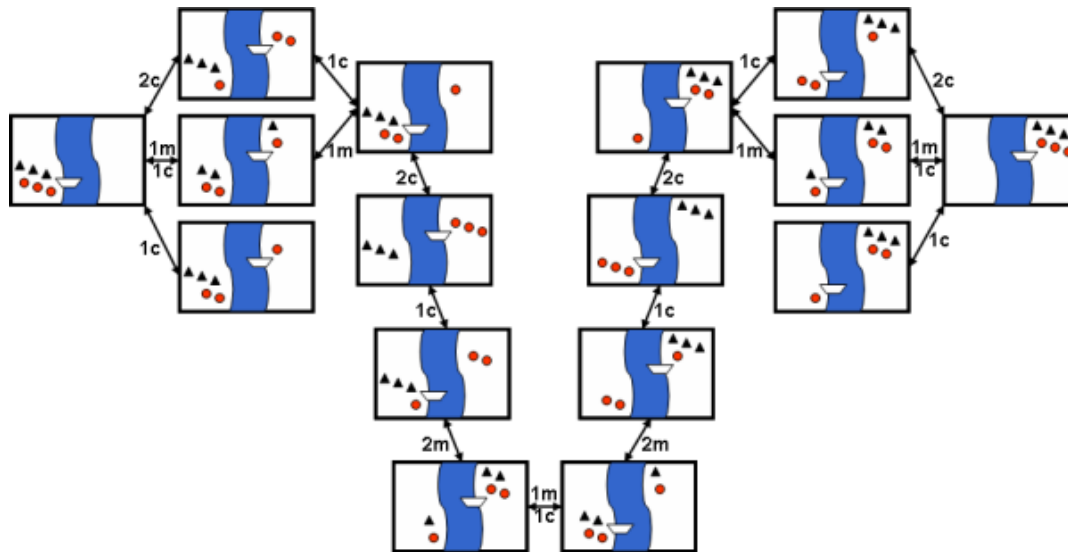
- Knuthova predpostavka: začeniši s številom 4 in zaporedjem treh operacij (faktoriela, kvadratni koren, zaokroževanje navzdol), lahko dosežemo poljubno pozitivno celo število. Npr. število 5 lahko zapišemo kot:

$$5 = \left\lfloor \sqrt{\sqrt{\sqrt{\sqrt{\sqrt{(4!)!}}}}} \right\rfloor$$

- začetno stanje** – število 4
- akcije** – uporaba ene izmed operacij *{faktoriela, kvadratni koren, zaokroževanje navzdol}*
- stanja** – pozitivna števila
- ciljno stanje** – želeno pozitivno število

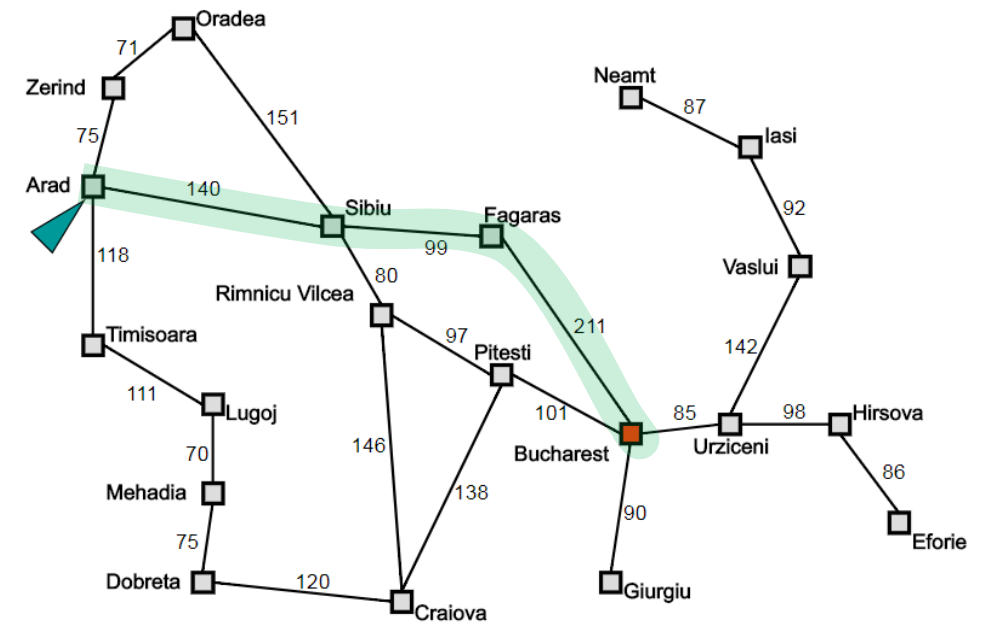
Primeri drugih umetnih problemov

- misijonarji in ljudožerci
- preurejanje postavitve kock



Primeri nekaterih realnih problemov

- iskanje najkrajše poti
- primer obhoda ("obišči vsa mesta vsaj enkrat, začnši v mestu X")
- problem potujočega trgovca - TSP ("najdi najkrajšo pot za obhod vseh mest natanko enkrat, začnši v mestu X")
- postavitve komponent na vezju (minimizacija površine, dolžine povezav itd.)
- navigacija robotov v prostoru
- sestavljanje izdelkov na proizvodni liniji

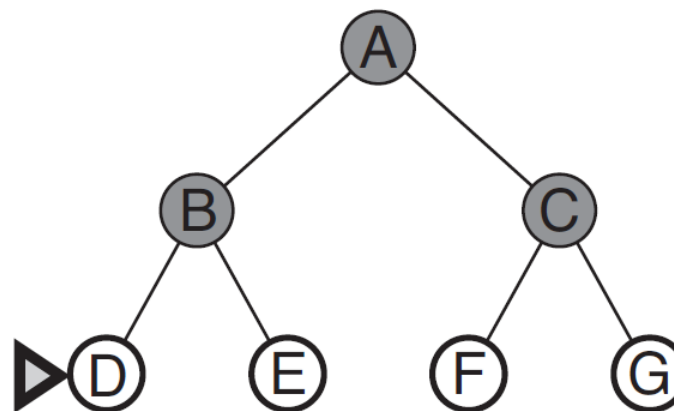
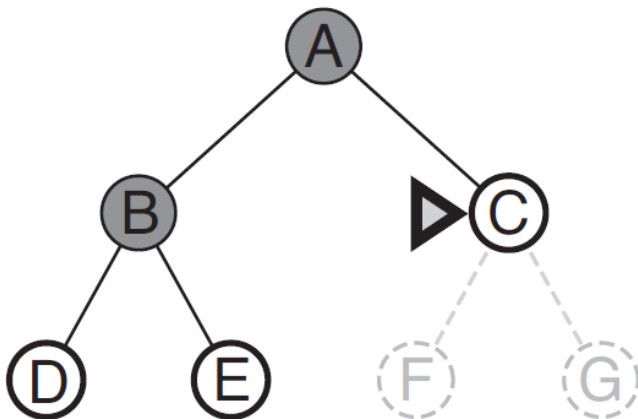
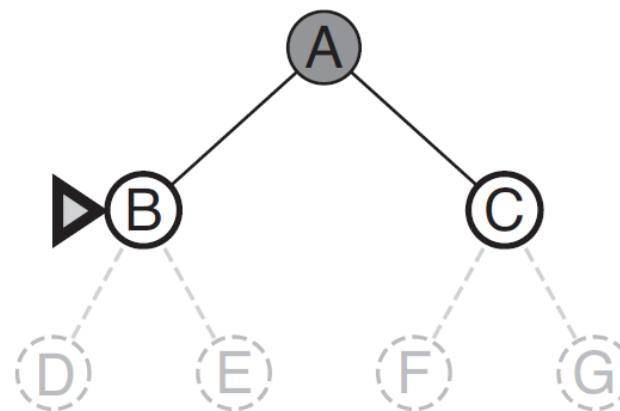
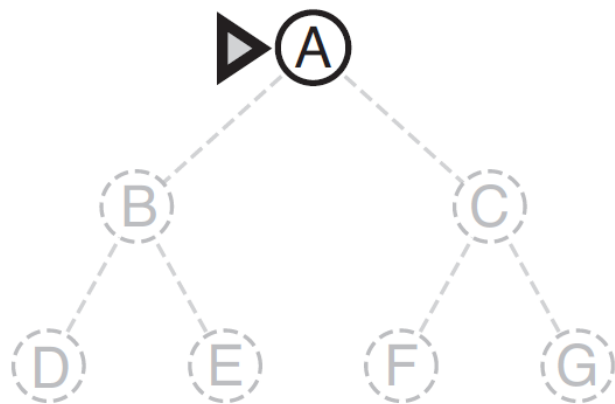


Preiskovanje

- problem: velika kombinatorična eksplozija možnih stanj
- preiskovalni algoritmi:
 - **neinformirani**: razpolagajo samo z definicijo problema
 - iskanje v širino (*breadth-first search*)
 - iskanje v globino (*depth-first search*)
 - iterativno poglobljanje (*iterative deepening*)
 - cenovno-optimalno iskanje (*uniform-cost search*)
 - **informirani**: razpolagajo tudi z dodatno informacijo (domensko znanje, hevristične ocene), kako bolj učinkovito najti rešitev
 - algoritem A*
 - algoritem IDA*
 - prioriteto preiskovanje (*best-first search*)
 - algoritem RBFS (*recursive best-first search*)
 - plezanje na hrib (*hill climbing*)
 - iskanje v snopu (*beam search*)
 - ...

Iskanje v širino

- angl. *Breadth-First Search (BFS)*
- strategija: vedno razvij najbolj **plitvo še nerazvito** vozlišče
 - implementacija: razvita vozlišča dodamo v vrsto (FIFO) za razvijanje



Iskanje v širino

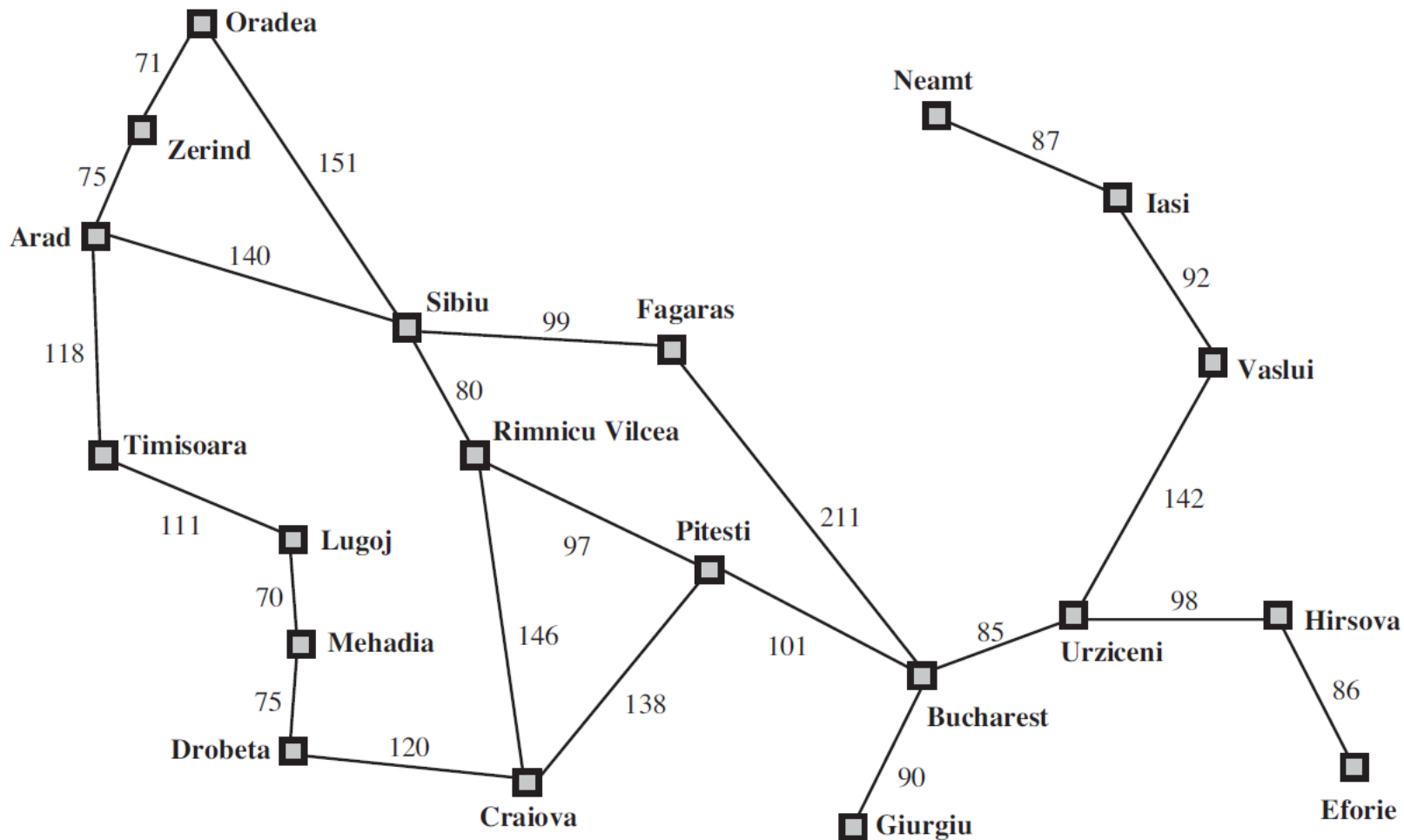
primer iz: Russell, Norvig, Artificial Intelligence: A Modern Approach, 3. izd., Pearson, 2010.

Romania



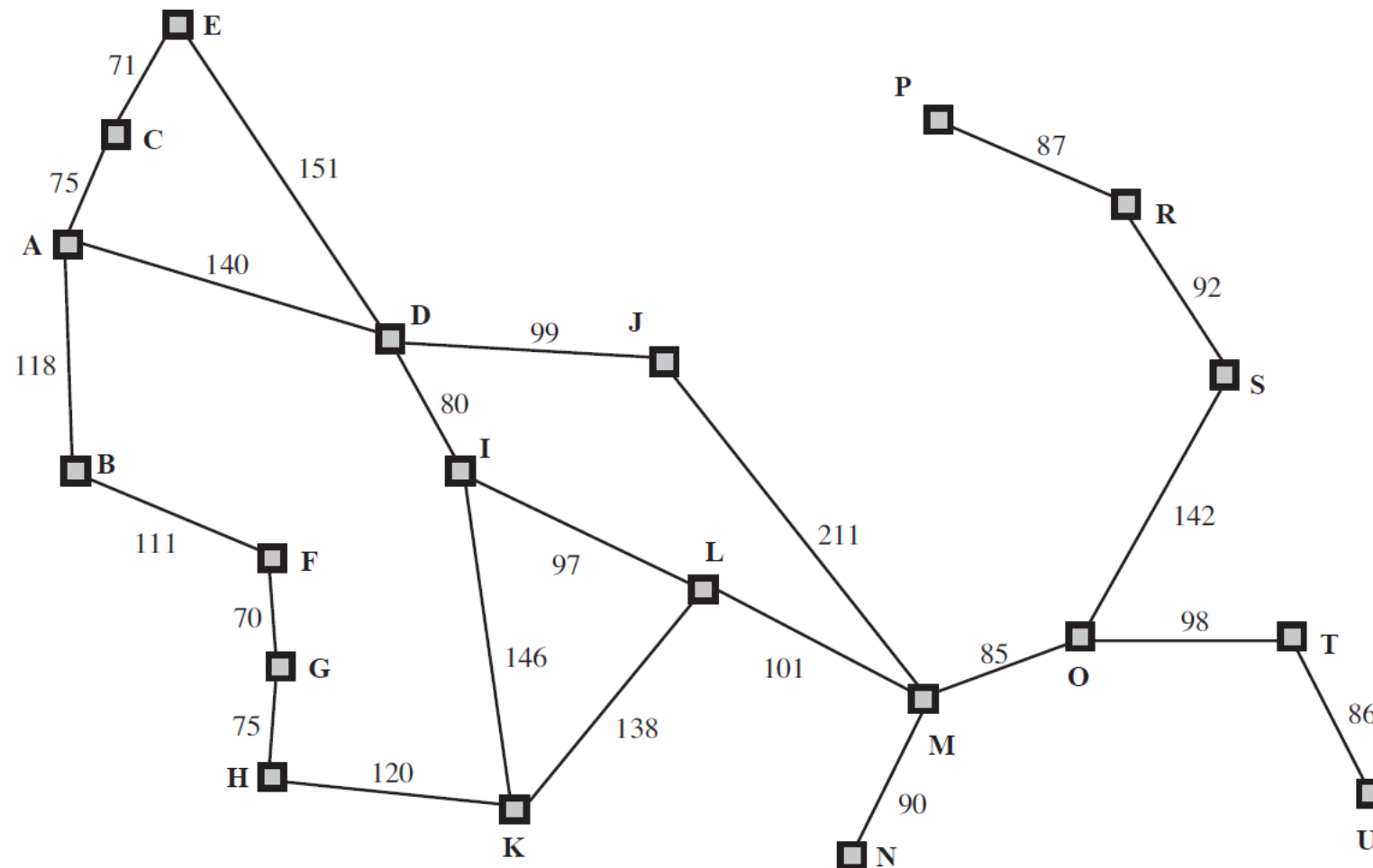
Iskanje v širino

primer iz: Russell, Norvig, Artificial Intelligence: A Modern Approach, 3. izd., Pearson, 2010.



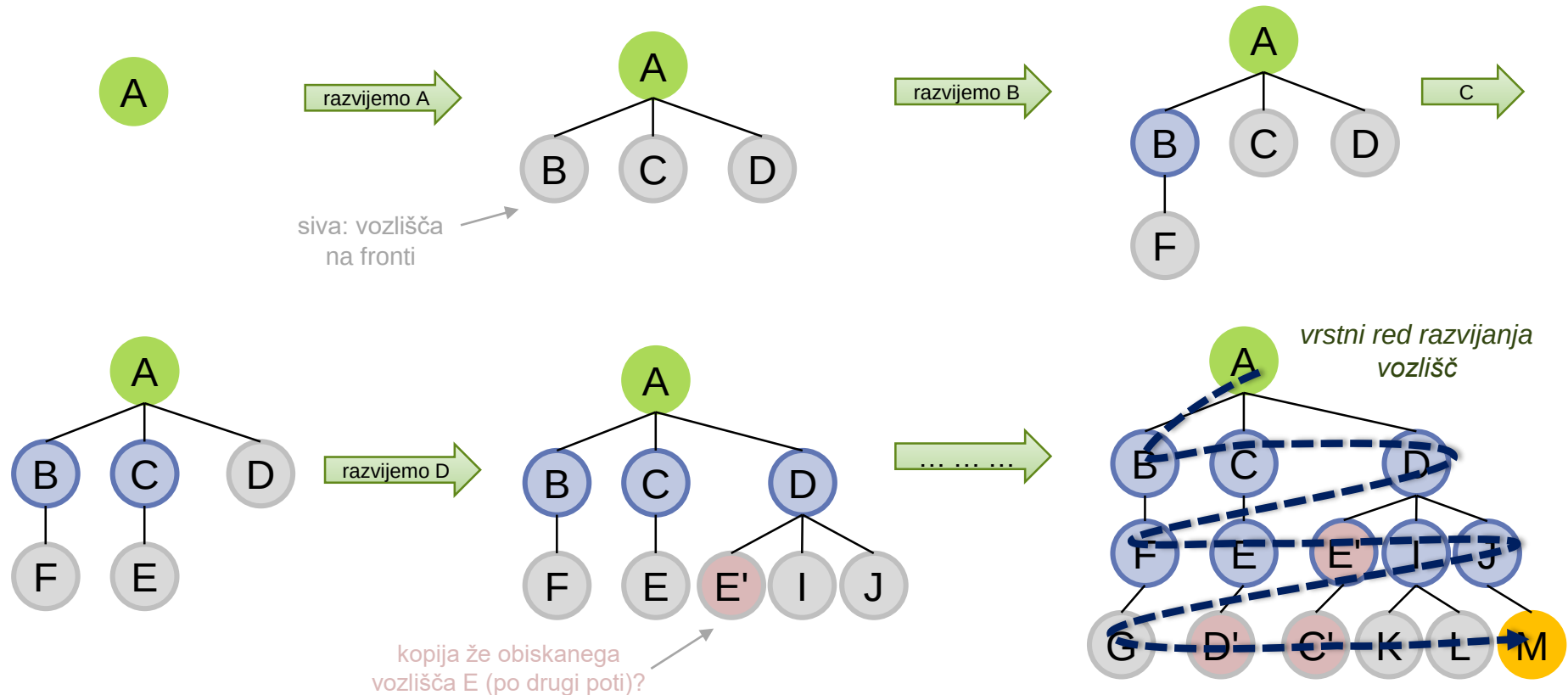
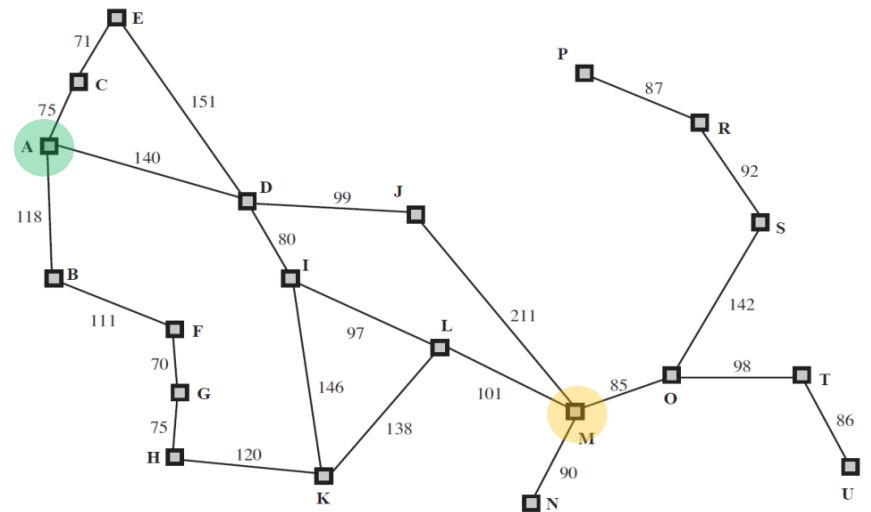
Iskanje v širino

primer iz: Russell, Norvig, Artificial Intelligence: A Modern Approach, 3. izd., Pearson, 2010.



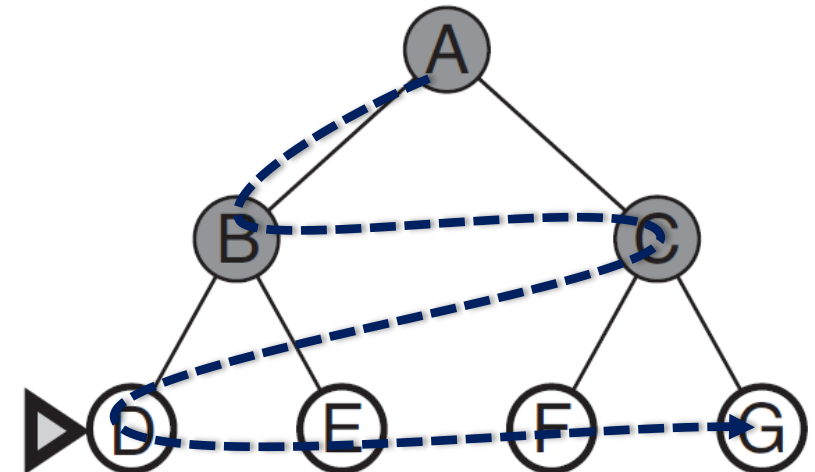
Iskanje v širino

- strategija: vedno razvij najbolj plitvo še nerazvito vozlišče
- odločimo se **hraniti kopije že obiskanih vozlišč (zakaj?)**
- **vozlišča, ki sklenejo cikel, takoj zavržemo (zakaj, je nujno?)**



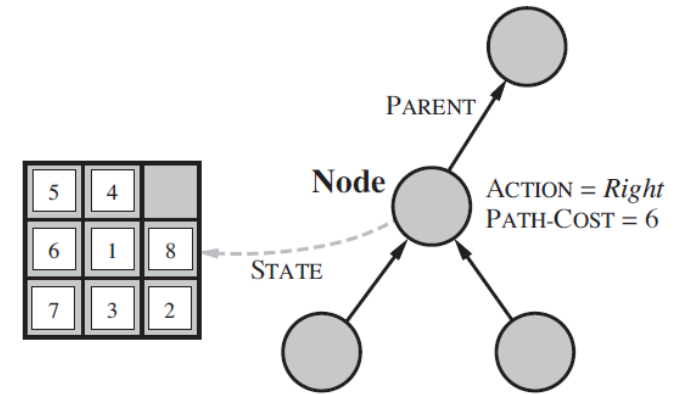
Iskanje v širino

- pojmi:
 - **razvijanje** vozlišča – **generiranje** naslednikov
 - **fronta** – listi drevesa, ki so kandidati za razvijanje (**FIFO vrsta**)
- generira celoten nivo preden se premakne na naslednji nivo
- v spominu **mora pomniti vse alternativne poti**, ki so kandidati, da bodo podaljšane do ciljnega vozlišča
- zagotavlja, da najdemo **najkrajšo rešitev** (A-D-J-M)
 - pozor, v grafu so možni cikli (več poti vodi do ciljnega vozlišča M)
- možne **implementacije in pomisleki**:
 - **detekcija ciljnega vozlišča**, dve možnosti:
 - ciljno vozlišče zaznamo šele, ko ga želimo razviti
 - ciljno vozlišče zaznamo, že ko ga generiramo (bolj optimalno)
 - **hranjenje kopij** že videnih vozlišč, da ali ne?
 - ne – pri BFS ni smiselno, ker imajo povezave enako ceno
 - da – kadar nas druga pot lahko pripelje do bolj optimalne rešitve (podane so različne cene povezav, potrebujemo algoritem, ki upošteva tudi cene na povezavah in ne samo vrstni red obiskovanja vozlišč; kasneje...)



Programska implementacija

- vozlišča v drevesu hranijo:
 - STANJE: opis stanja v problemskem prostoru
 - PREDHODNIK: kazalec na predhodnika
 - AKCIJA: akcija, ki je iz predhodnika generirala vozlišče
 - CENA: cena poti od začetnega do trenutnega vozlišča

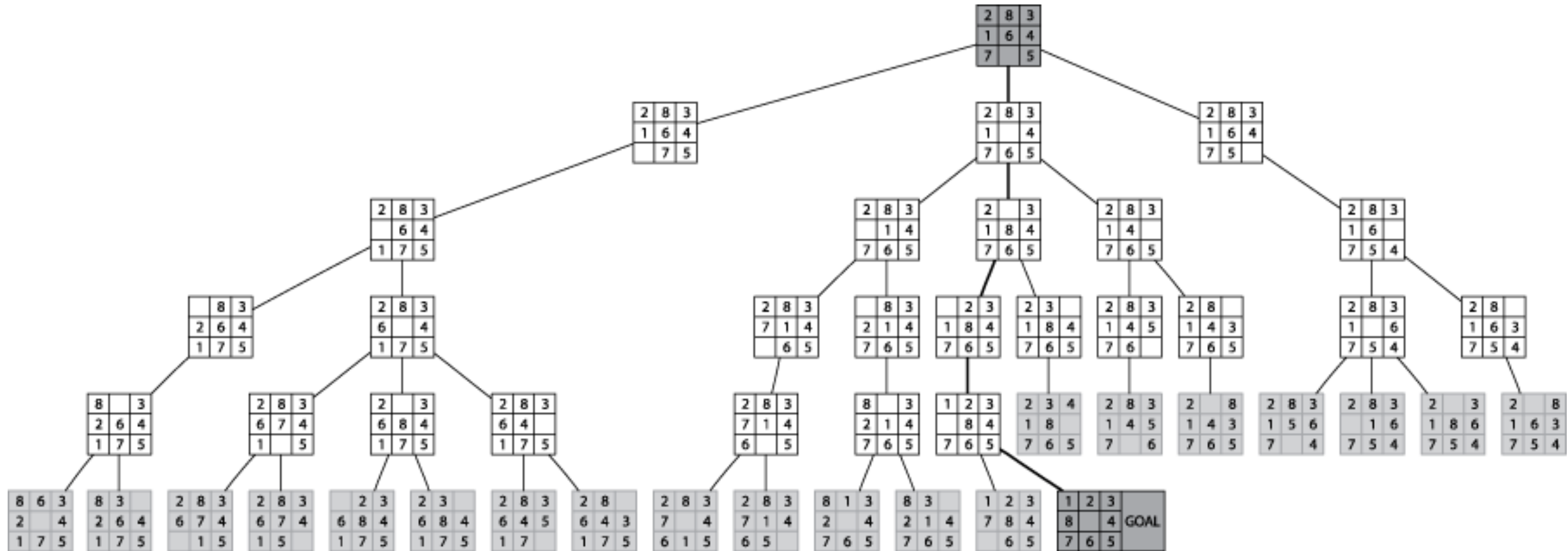


```
function iskanje(problem) {  
  vozlišče <- problem.začetno_stanje, cena=0  
  if (cilj(vozlišče.stanje)) return rešitev=vozlišče  
  fronta <- vozlišče  
  obiskana <- ∅  
  while(true) {  
    if (fronta== ∅) return rešitev= ∅  
    vozlišče <- izberi(fronta)  
    fronta <- fronta - vozlišče  
    obiskana <- obiskana + vozlišče.stanje  
    foreach naslednik in nasledniki(vozlišče) {  
      if ((naslednik.stanje ∉ obiskana) && (naslednik ∉ fronta))  
        if (cilj(naslednik.stanje) return rešitev=naslednik  
        fronta <- fronta + naslednik  
    }  
  }  
}
```

*način dela s fronto določa strategijo
preiskovalnega algoritma*

*hranjenje obiskanih stanj, do
katerih lahko pridemo po različnih
poteh, omogoča preprečitev
ciklanja preiskovanja*

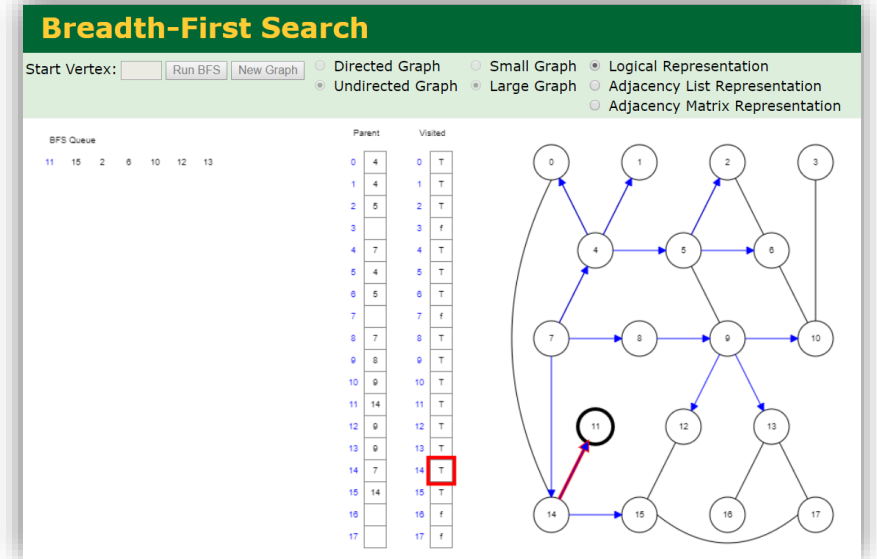
Primer: igra 8 ploščic



Demo

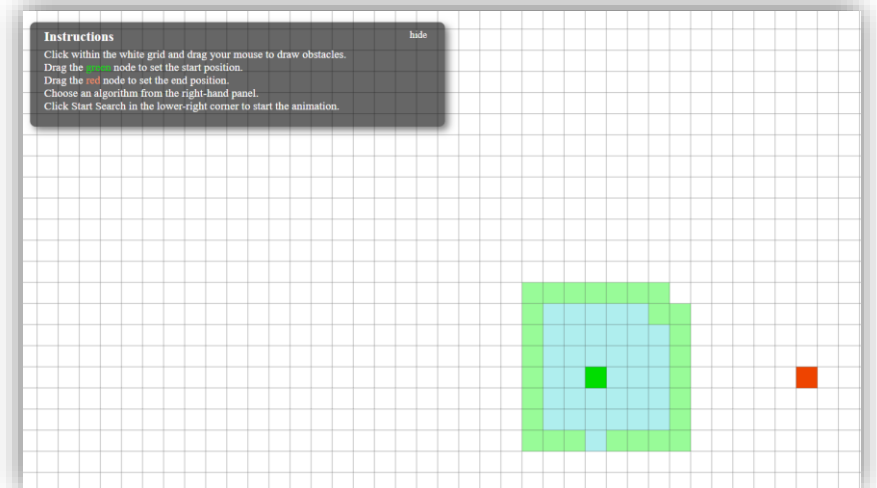
- Data Structure Visualizations

<https://www.cs.usfca.edu/~galles/visualization/BFS.html>



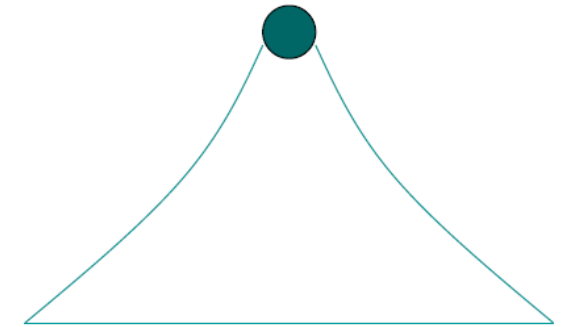
- PathFinding

<https://qiao.github.io/PathFinding.js/visual/>



Učinkovitost iskanja v širino

- za potrebe analize predpostavimo, da je prostor stanj drevo višine d (*depth*) in stopnje vejanja b (*branching factor*)
 - na nivoju d imamo torej b^d vozlišč
- **POPOLNOST** (angl. *completeness*):
Ali algoritem zagotovo najde rešitev, če le-ta obstaja?
 - DA! (če je le b končen)
- **OPTIMALNOST** (angl. *optimality*):
Ali iskanje najde optimalno (najboljšo možno rešitev)?
 - BFS da (če je optimalna najkrajša pot), vendar v splošnem (cena povezav ni 1) to ni nujno.
- **ČASOVNA ZAHTEVNOST**:
 - generirano število vozlišč je $b + b^2 + b^3 + \dots + b^d = O(b^d)$
 - torej: eksponentna časovna zahtevnost glede na d
- **PROSTORSKA ZAHTEVNOST**:
 - hraniti mora $O(b^{d-1})$ razvitih vozlišč in $O(b^d)$ v fronti – skupaj $O(b^d)$



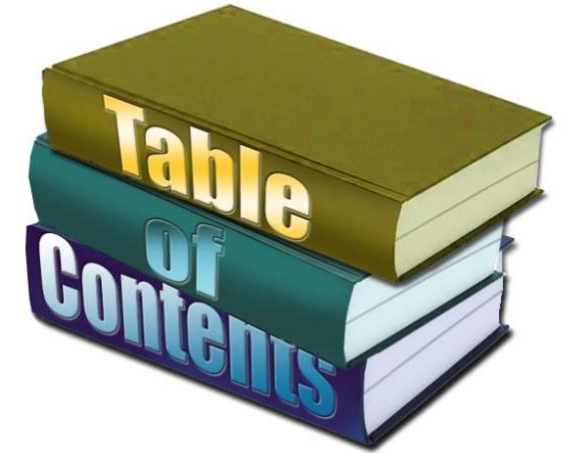
Zahtevnost iskanja v širino

- primer za faktor vejanja $b = 10$, hitrost generiranja 10^6 vozlišč/sekundo, potreben prostor 1000 bajtov/vozlišče

Globina	Vozlišč	Čas	Spomin
2	110	0,11 ms	107 kB
4	11.110	11 ms	10,6 MB
6	10^6	1,1 s	1 GB
8	10^8	2 minuti	103 GB
10	10^{10}	3 ure	10 TB
12	10^{12}	13 dni	1 PB
14	10^{14}	3,5 let	99 PB
16	10^{16}	350 let	10 EB

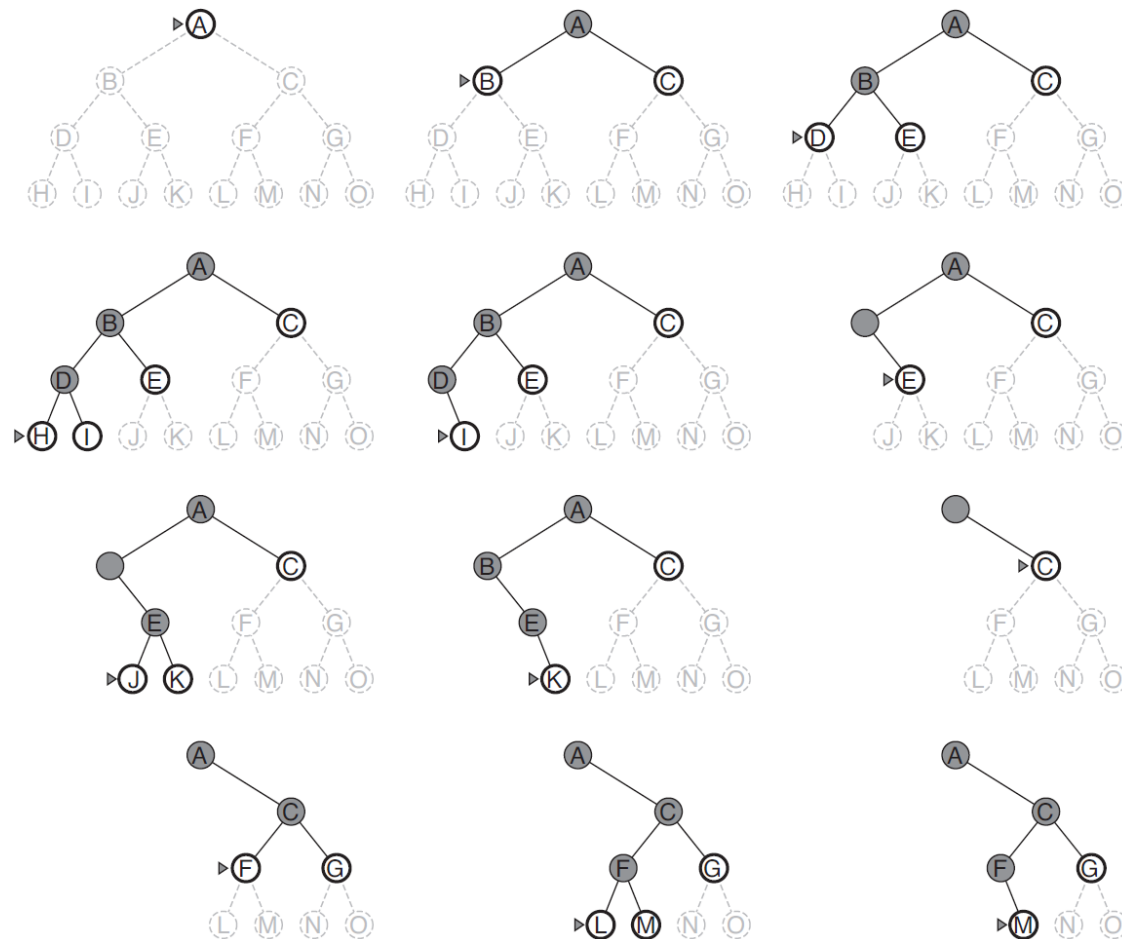
Pregled

- preiskovanje prostora stanj
 - definicija in cilji področja
 - primeri umetnih in realnih problemov
 - neinformirani preiskovalni algoritmi
 - iskanje v širino
 - iskanje v globino
 - iterativno poglobljanje
 - dvosmerno iskanje
 - cenovno – optimalno iskanje
 - informirani preiskovalni algoritmi
 - hevristično preiskovanje
 - požrešno preiskovanje
 - A*
 - IDA*
 - kakovost hevrističnih funkcij



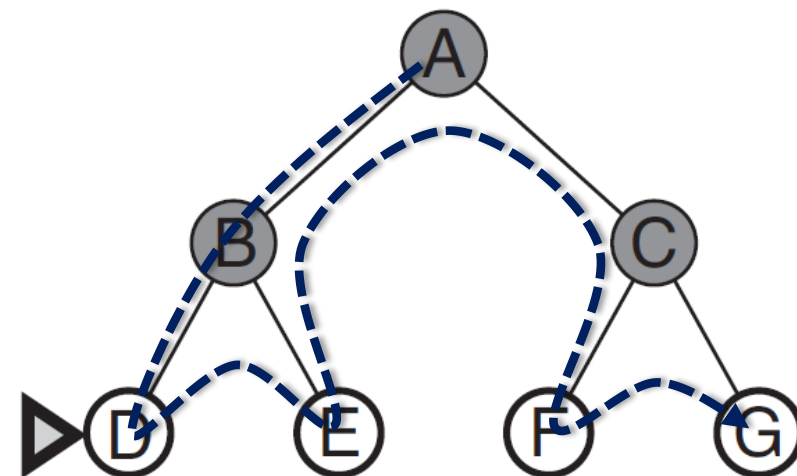
Iskanje v globino

- angl. *Depth-First Search (DFS)*
- strategija: najprej razvij **najgloblje še nerazvito** vozlišče
 - implementacija: naslednike dodaj v sklad (LIFO) za razvijanje



Iskanje v globino

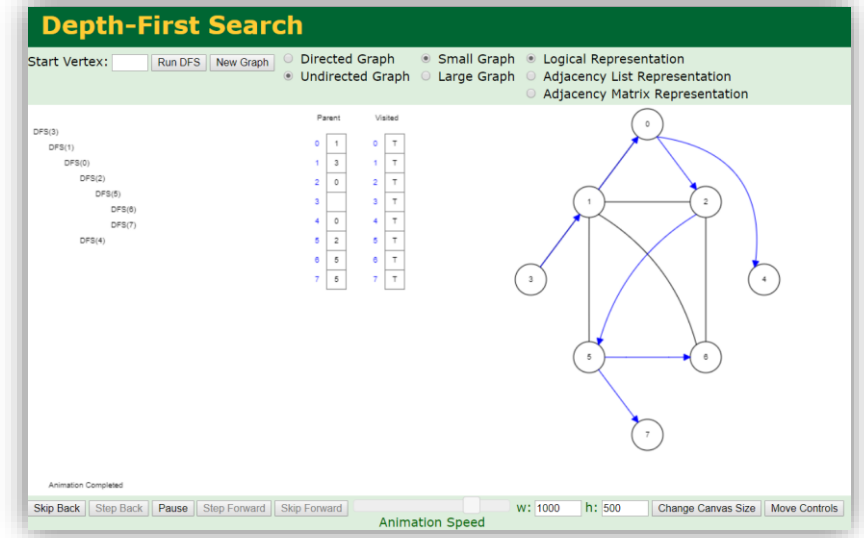
- ne zagotavlja najkrajše rešitve
- nujno potrebno preprečevanje ciklov, možnost neskončno globokega prostora (neskončna globina)
- možna je preprosta implementacija z rekurzijo
- možna optimizacija: raziskanih vej nam ni treba hraniti v spominu (hranimo le pot od začetnega do trenutnega vozlišča)



Demo

- Data Structure Visualizations

<https://www.cs.usfca.edu/~galles/visualization/DFS.html>

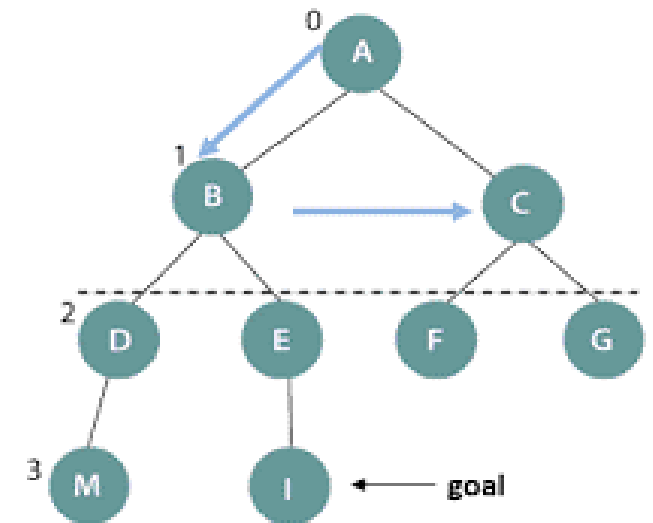


Učinkovitost iskanja v globino

- za potrebe analize predpostavimo, da je prostor stanj drevo:
 - globina (*depth*) optimalne rešitve naj bo **d**
 - stopnja vejanja (*branching factor*) naj bo **b**
na nivoju d imamo torej b^d vozlišč
 - največja globina drevesa naj bo **max**
- **POPOLNOST** (angl. *completeness*):
 - Neuspešen v prostorih z zankami in neskončno globino.
- **OPTIMALNOST** (angl. *optimality*):
 - Ne.
- **ČASOVNA ZAHTEVNOST**:
 - generirano število vozlišč $O(b^{max})$
- **PROSTORSKA ZAHTEVNOST**:
 - hraniti mora samo $O(bm)$ razvitih vozlišč (linearna prostorska zahtevnost!)

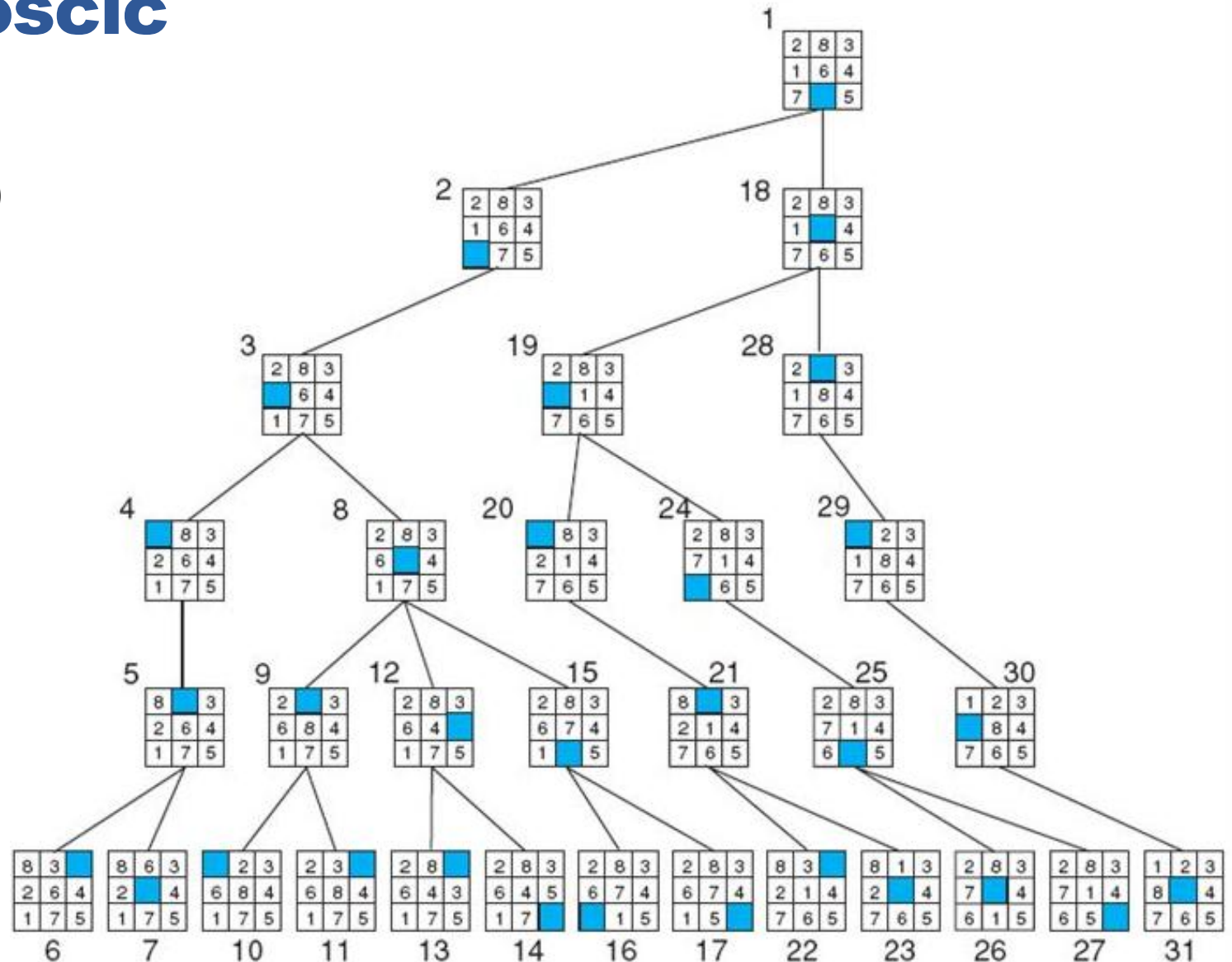
Iskanje v globino - izboljšave

- iskanje s sestopanjem (*backtracking search*):
 - namesto vseh naslednikov generiramo samo enega po enega
 - ➔ prostorska zahtevnost $O(m)$
- iskanje z omejitvijo globine (*depth-limited search*):
 - vnaprej definiramo mejo globine l
 - vozlišča na globini l obravnavamo, kot da nimajo naslednikov
 - če izberemo $l < d$, je algoritem nepopoln (ne najde rešitve)
 - če izberemo $l > d$, je algoritem popoln, a neoptimalen
 - časovna zahtevnost je $O(b^l)$, prostorska pa $O(bl)$
 - pri določitvi l pomaga domensko znanje
- iterativno poglobljanje (angl. *Iterative Deepening Search (IDS)*)



Primer: igra 8 ploščic

- iskanje z omejitvijo globine (d=6)



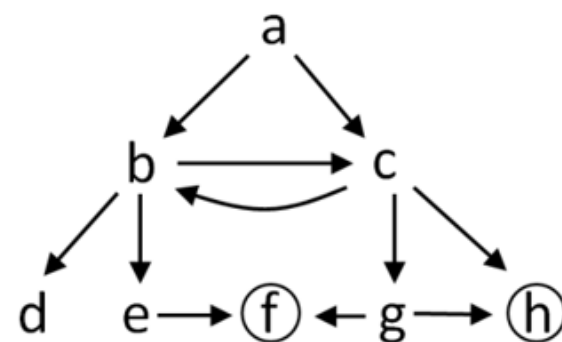
Goal

Naloga

- 1. izpit, 30. 1. 2018, 1. naloga

1. NALOGA:

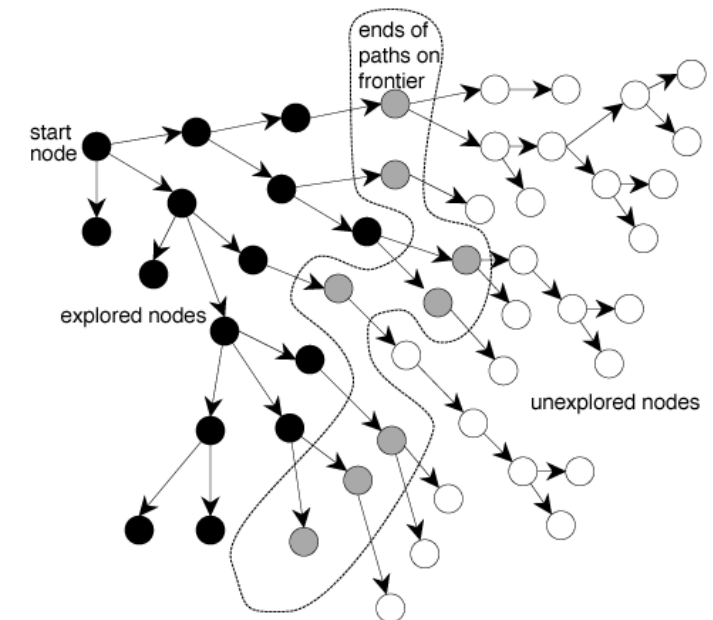
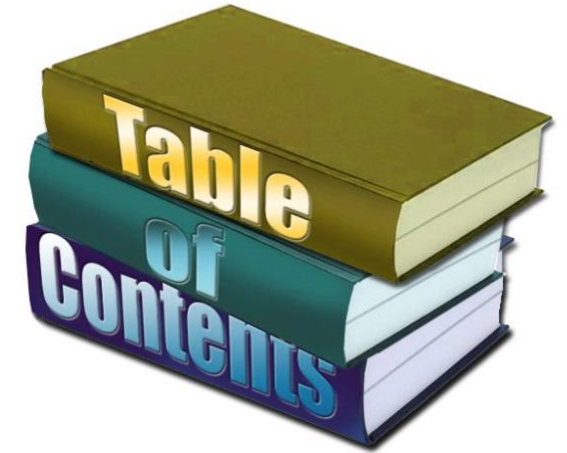
Naj bo a začetno vozlišče preiskovanja, f in h pa sta ciljni vozlišči. Algoritmi preiskovanja naj generirajo naslednike vozlišč po abecednem vrstnem redu. Npr. vrstni red naslednikov vozlišča c je: b, g, h . Vsi preiskovalni algoritmi preverjajo pogoj, ali je dano vozlišče ciljno šele ob času, ko se lotijo razvijanja tega vozlišča. Vsi preiskovalni algoritmi naj tudi razpoznavajo cikle in generirano vozlišče, ki sklene cikel, takoj zavržejo. Vendar pa obravnavajo graf kot drevo. Torej, če pridejo do kakega vozlišča N po različnih poteh, naredijo kopijo N' vozlišča N in obravnavajo N' , kot da bi bilo novo vozlišče. Če imata dve vozlišči enako f -oceno, se najprej razvije tisto vozlišče, ki je bilo prej generirano.



- Katero rešitveno pot vrne **iskanje v globino**? Kakšen je vrstni red generiranih vozlišč?
- Katero rešitveno pot vrne **iskanje v širino**? Kakšen je vrstni red generiranih vozlišč?

Pregled

- preiskovanje prostora stanj
 - definicija in cilji področja
 - primeri umetnih in realnih problemov
 - neinformirani preiskovalni algoritmi
 - iskanje v širino
 - iskanje v globino
 - iterativno poglobljanje
 - dvosmerno iskanje
 - cenovno – optimalno iskanje
 - informirani preiskovalni algoritmi
 - hevristično preiskovanje
 - požrešno preiskovanje
 - A*
 - IDA*
 - kakovost hevrističnih funkcij



Iterativno poglobljanje

- problem globinsko omejenega iskanja v globino je nastavitev meje l
- rešitev: iterativno poglobljanje (*iterative deepening depth-first search*)
- strategija: začnimo z nizko mejo *limit* in jo povečujemo za 1, dokler ne najdemo rešitve. Na vsakem koraku poženimo iskanje v globino.

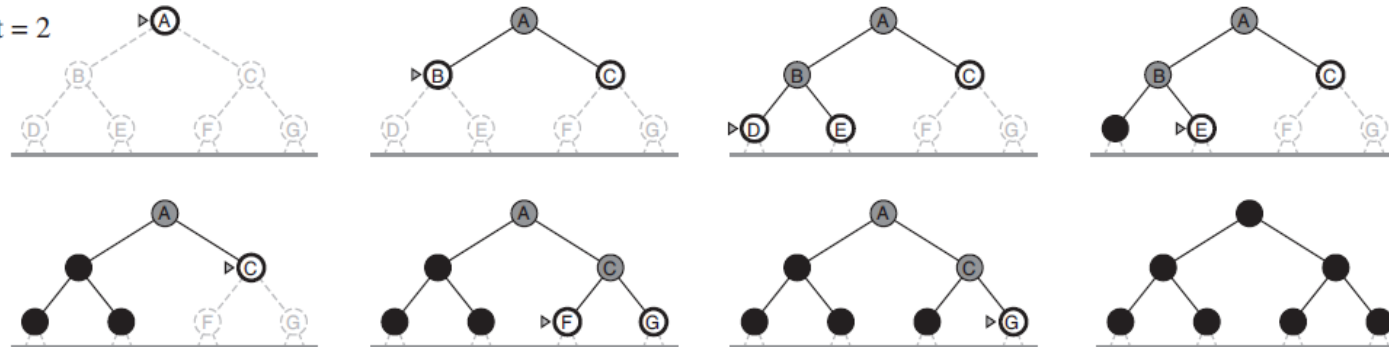
Limit = 0



Limit = 1

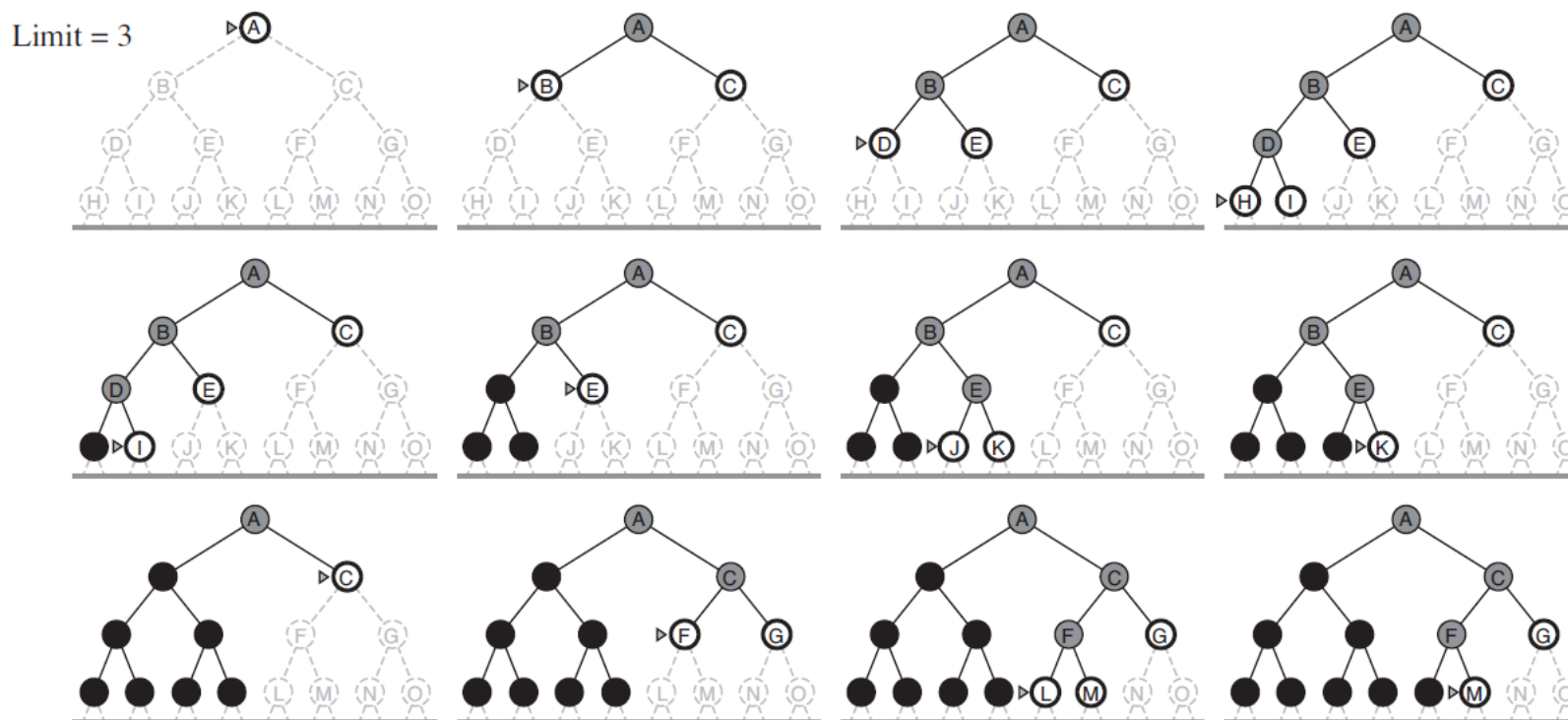


Limit = 2



Iterativno poglabljanje

- problem globinsko omejenega iskanja v globino je nastavitev meje l
- rešitev: iterativno poglabljanje (*iterative deepening depth-first search*)
- strategija: začnimo z nizko mejo *limit* in jo povečujemo za 1, dokler ne najdemo rešitve. Na vsakem koraku poženimo iskanje v globino.



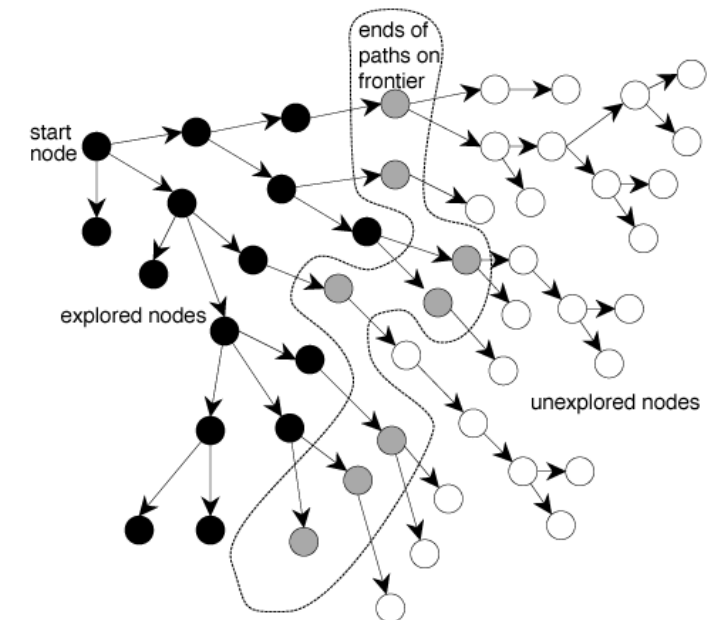
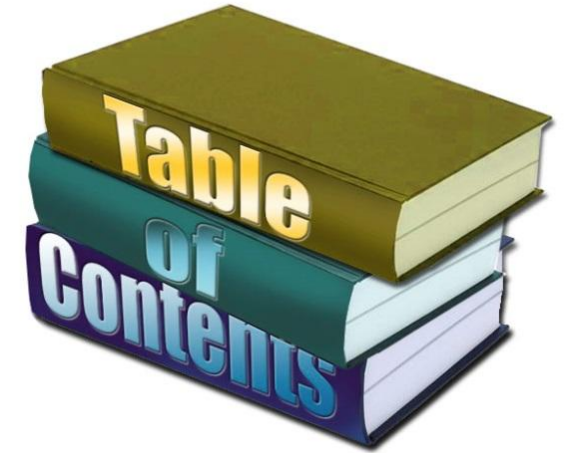
Učinkovitost iterativnega poglabljanja

- **POPOLNOST** (angl. *completeness*):
 - Da.
- **OPTIMALNOST** (angl. *optimality*):
 - Da (v kolikor iščemo najkrajšo rešitev).
- **ČASOVNA ZAHTEVNOST:**
 - v iteracijah se ponavlja generiranje istih vozlišč znova
 - asimptotično gledano, je generirano število vozlišč enako kot pri iskanju v širino:
$$[b] + [b + b^2] + \dots [b + b^2 + \dots + b^d]$$
$$= db + (d - 1)b^2 + \dots + 2b^{d-1} + b^d = O(b^d)$$
 - kljub višji ceni pa je ta cena še vedno sprejemljiva, ker se največ vozlišč generira na zadnjem nivoju (npr. za $b = 10$, $d = 5$, velja: $N(IDS) = 123.450$, $N(BFS) = 111.110$)
- **PROSTORSKA ZAHTEVNOST:**
 - hraniti mora samo $O(bd)$ razvitih vozlišč (linearna prostorska zahtevnost!)
- **metoda torej kombinira prednosti iskanja v širino (popolnost, optimalnost) in iskanja v globino (linearna prostorska zahtevnost)**

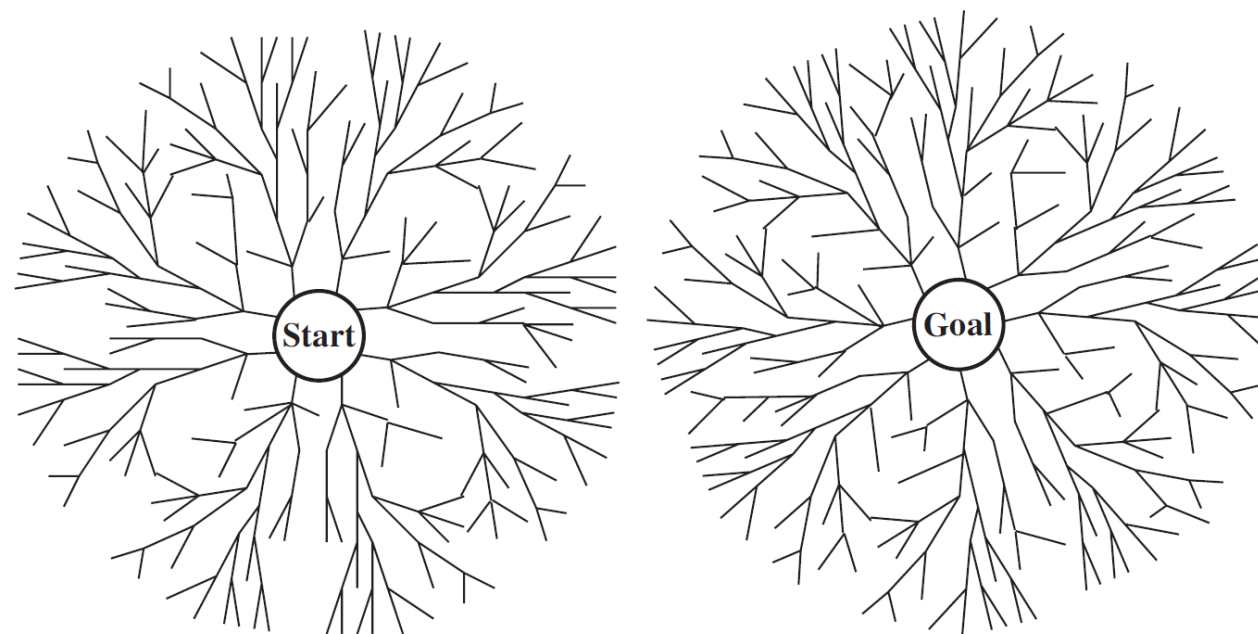


Pregled

- preiskovanje prostora stanj
 - definicija in cilji področja
 - primeri umetnih in realnih problemov
 - neinformirani preiskovalni algoritmi
 - iskanje v širino
 - iskanje v globino
 - iterativno poglobljanje
 - dvosmerno iskanje
 - cenovno – optimalno iskanje
 - informirani preiskovalni algoritmi
 - hevristično preiskovanje
 - požrešno preiskovanje
 - A*
 - IDA*
 - kakovost hevrističnih funkcij



Dvosmerno iskanje

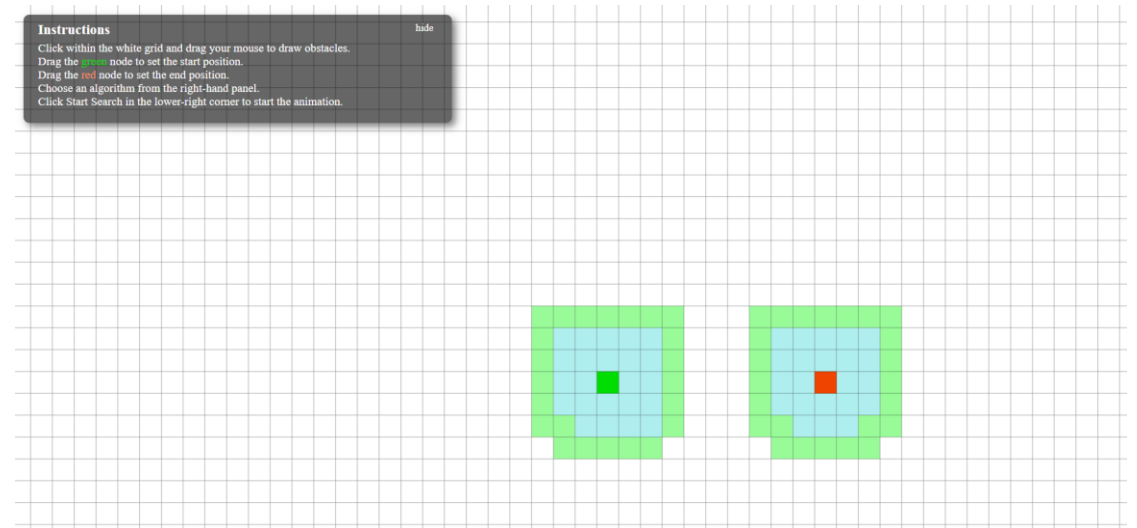


- ideja: pognati vzporedni iskanji od začetnega vozlišča proti cilju in vzvratno od cilja proti začetnemu vozlišču z upanjem, da se iskanji "srečata" na polovici poti
- motivacija: zaradi znižanja globine iskanja želimo doseči časovno zahtevnost $b^{d/2} + b^{d/2} = O(b^{d/2})$, kar je manj kot $O(b^d)$

Demo

- PathFinding

<https://qiao.github.io/PathFinding.js/visual/>



Implementacija dvosmernega iskanja

- za izvedbo vzvratnega iskanja morajo vozlišča imeti kazalec na predhodnika
- ciljno vozlišče mora biti znano
 - pri igri 8 ploščic je npr. znano, pri uganki Sudoku pa ne
- uporabimo lahko poljuben preiskovalni algoritem
 - če uporabimo iskanje v širino, najde algoritem optimalno rešitev
- cilj iskanja preverja, ali obstaja med frontama obeh iskanj presečišče
- problemski prostor lahko redefiniramo tako, da en korak iskanja v "dvosmernem" prostoru predstavlja dva koraka (od začetka proti cilju in od cilja proti začetku) v originalnem prostoru
 - če v originalnem prostoru velja



potem definiramo v novem prostoru novi vozlišči (S, E) in $(S1, E1)$

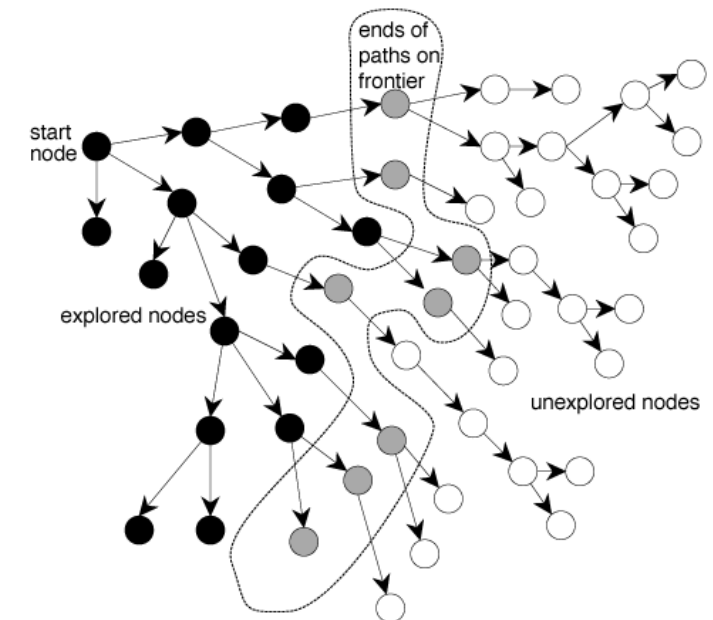
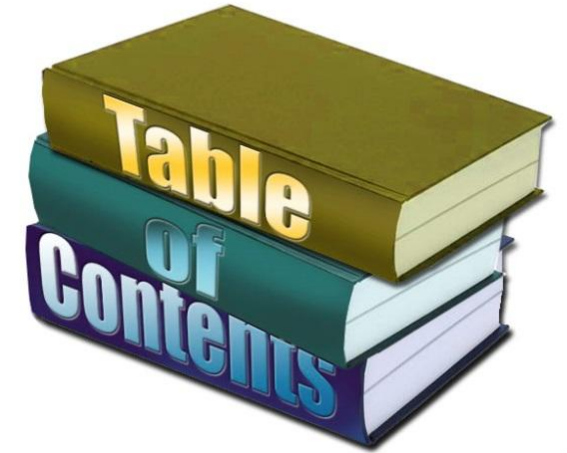
- v "dvosmernem" prostoru velja $(S, E) \rightarrow (S1, E1)$, če obstaja v "enosmernem" prostoru povezava med $S \rightarrow S1$ in med $E1 \rightarrow E$
- vozlišče (S, E) je v "dvosmernem" prostoru ciljno vozlišče, če velja $E=S$ ali $S \rightarrow E$

Primerjava časovnih zahtevnosti

Kriterij	Iskanje v širino	Iskanje v globino	Iskanje z omejitvijo globine	Iterativno poglobljanje	Dvosmerno iskanje
Popolnost	Da (če je b končen)	Ne	Ne	Da (če je b končen)	Da
Optimalnost	Da (če so cene enake)	Ne	Ne	Da (če so cene enake)	Da
Čas. zahtevnost	$O(b^d)$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
Prost. zahtevnost	$O(b^d)$	$O(bm)$	$O(bl)$	$O(bd)$	$O(b^{d/2})$

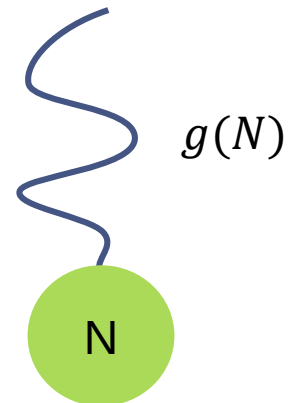
Pregled

- preiskovanje prostora stanj
 - definicija in cilji področja
 - primeri umetnih in realnih problemov
 - neinformirani preiskovalni algoritmi
 - iskanje v širino
 - iskanje v globino
 - iterativno poglobljanje
 - dvosmerno iskanje
 - cenovno – optimalno iskanje
 - informirani preiskovalni algoritmi
 - hevristično preiskovanje
 - požrešno preiskovanje
 - A*
 - IDA*
 - kakovost hevrističnih funkcij



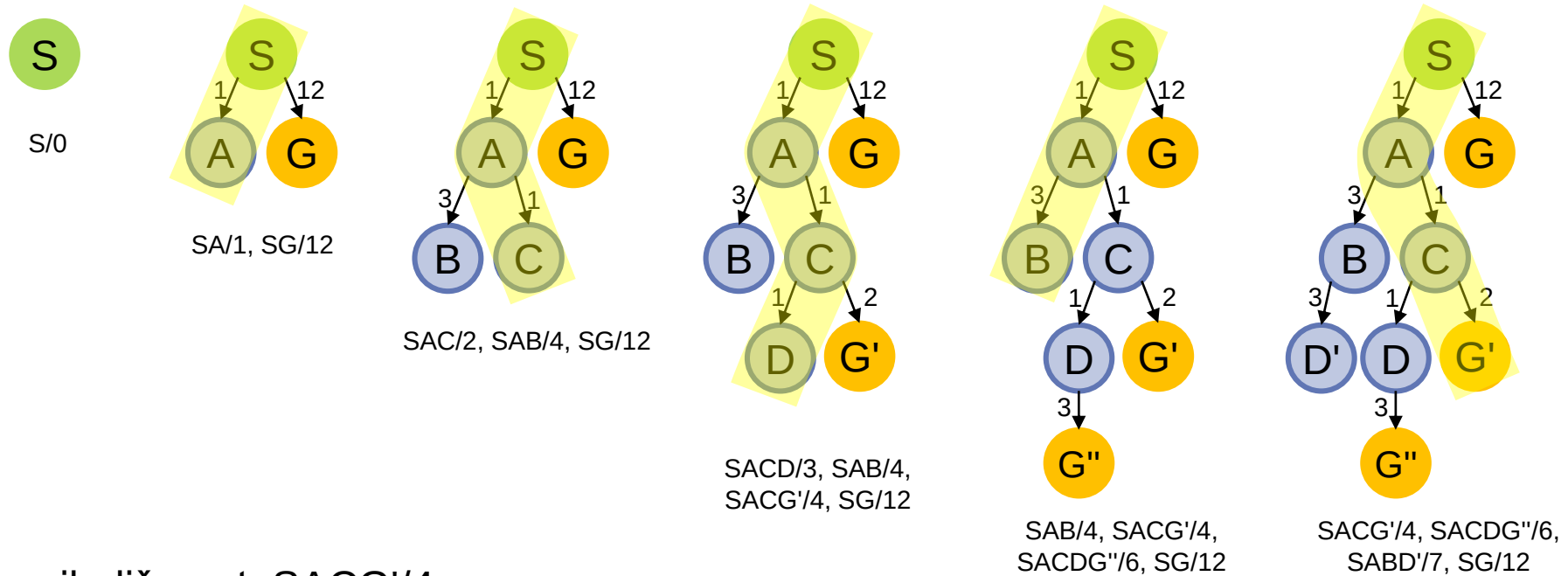
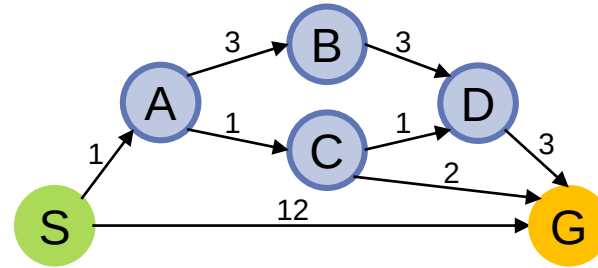
Cenovno-optimalno iskanje

- angl. *uniform-cost search (best-first search with no heuristic)*
- posplošitev **iskanja v širino**
 - iskanje v širino je optimalno, če so cene vseh povezav enake 1
- če cene vseh povezav **niso enake**, je optimalno **razviti vozlišče**, ki ima **najmanjšo skupno ceno dosedanje poti** – $g(n)$
- fronta je urejena kot prioritetna vrsta
- test, ali je vozlišče ciljno, opravimo šele, ko je vozlišče na vrsti za razvijanje in ne ob generiranju vozlišča
 - zakaj?
 - ciljno vozlišče morda ni optimalno (obstaja boljša rešitev)
 - morda do najdenega optimalnega cilja vodi krajša pot



Cenovno-optimalno iskanje

- primer



- rešitev: najboljša pot: SACG'/4

Učinkovitost iskanja

- za potrebe analize predpostavimo, da je prostor stanj drevo:
 - globina (*depth*) optimalne rešitve naj bo d
 - stopnja vejanja (*branching factor*) naj bo b , na nivoju d imamo torej b^d vozlišč
 - največja globina drevesa naj bo max
 - C^* naj bo cena optimalne rešitve
 - ϵ naj bo najmanjša cena povezave
- **POPOLNOST** (angl. *completeness*):
 - Da, za cene povezav > 0 .
- **OPTIMALNOST** (angl. *optimality*):
 - Da.
- **ČASOVNA in PROSTORSKA ZAHTEVNOST:**
 - odvisni sta od cen poti in ne samo od globine d in vejanja b
 - zahtevnost $O(b^{1+\lceil C^*/\epsilon \rceil})$, kar je lahko veliko več kot $O(b^d)$
 - če so vse cene poti enake, se zahtevnost poenostavi v $O(b^{1+d})$
 - zakaj $O(b^{d+1})$ in ne $O(b^d)$?

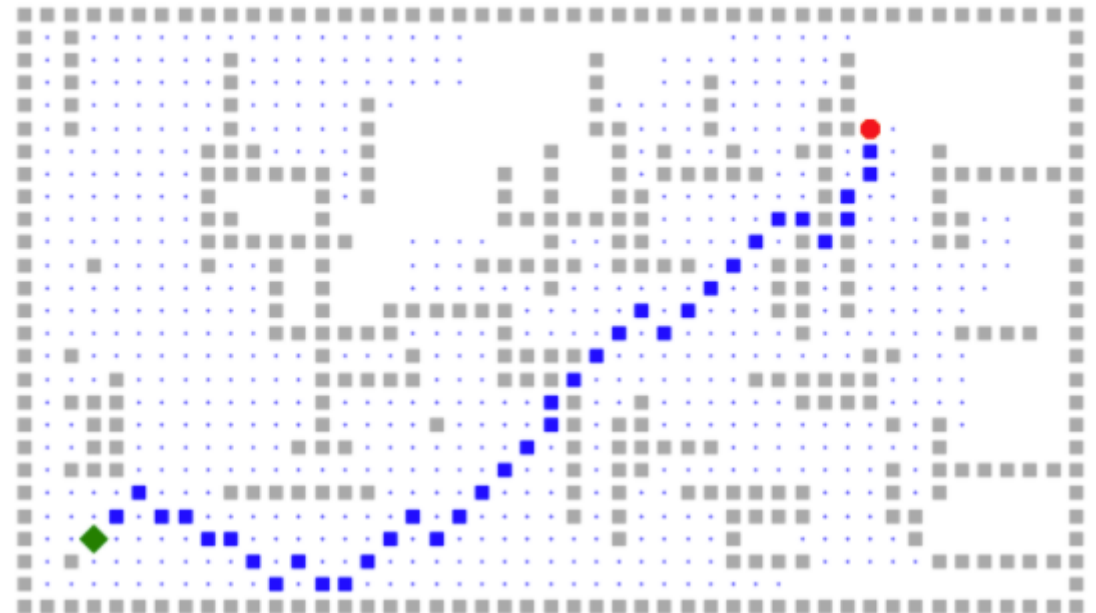


Demo

- Searching in AI

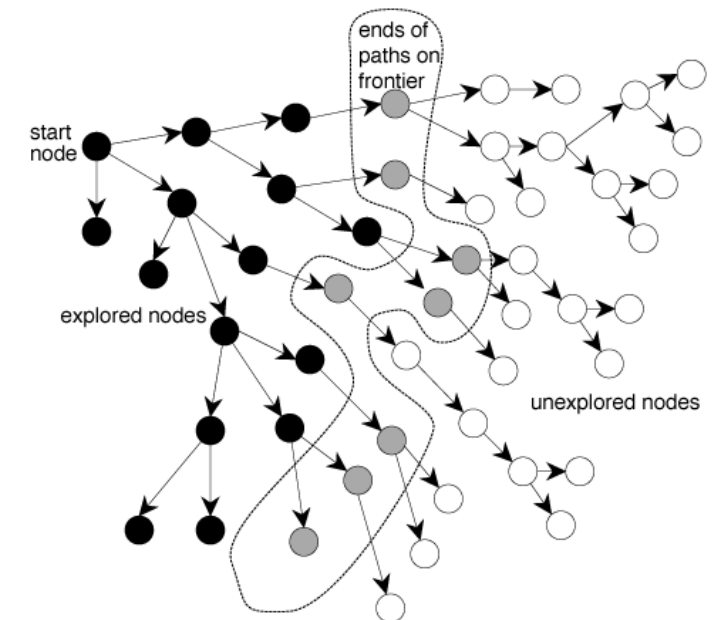
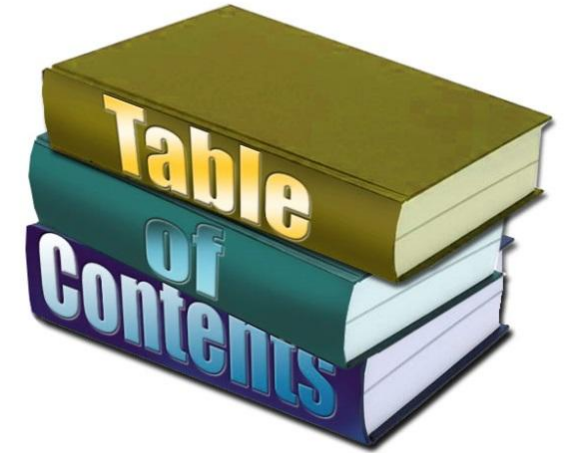
<https://medium.com/analytics-vidhya/searching-in-ai-e05973068c8e>

Explored: 661, Path Cost: 218, Memory: 70.148096MiB
Solver: astar



Pregled

- preiskovanje prostora stanj
 - definicija in cilji področja
 - primeri umetnih in realnih problemov
 - neinformirani preiskovalni algoritmi
 - iskanje v širino
 - iskanje v globino
 - iterativno poglobljanje
 - dvosmerno iskanje
 - cenovno – optimalno iskanje
 - informirani preiskovalni algoritmi
 - hevristično preiskovanje
 - požrešno preiskovanje
 - A*
 - IDA*
 - kakovost hevrističnih funkcij

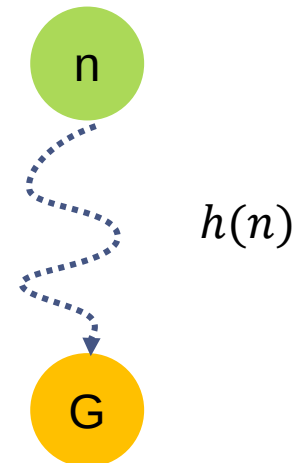


Hevristično preiskovanje

- potreba po usmerjanju iskanja z motivacijo, da hitreje/lažje najde optimalno rešitev
- ideja: uporabimo **oceno vozlišč** (stanj), ki ocenjujejo obetavnost za doseganje do cilja
- **hevrstika** (ali hevrstična ocena, ugibanje) je **ocenitvena funkcija za obetavnost vozlišča** (najcenejše poti iz vozlišča do najbližjega cilja)

$$h: \text{vozlišče} \rightarrow \mathbb{R}$$

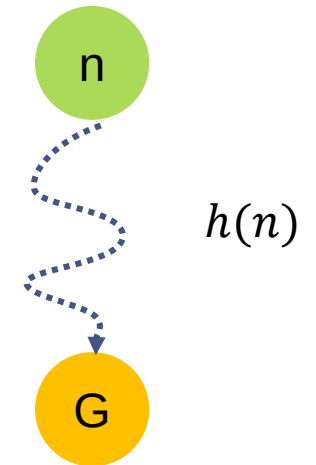
- izberemo in razvijemo vozlišče glede na **najboljšo vrednost hevrstike**
 - nizek h nakazuje **bolj obetavno** vozlišče, višji h pa **manj obetavno** (težji problem)
 - fronta hrani vozlišča, urejena v **prioritetni vrsti po obetavnosti**
- primeri hevrstičnih preiskovalnih algoritmov:
 - A^*
 - IDA* (*iterative deepening A**)
 - RBFS (*recursive best-first search*)
 - iskanje v snopu (*beam search*)
 - plezanje na hrib (*hill climbing*)



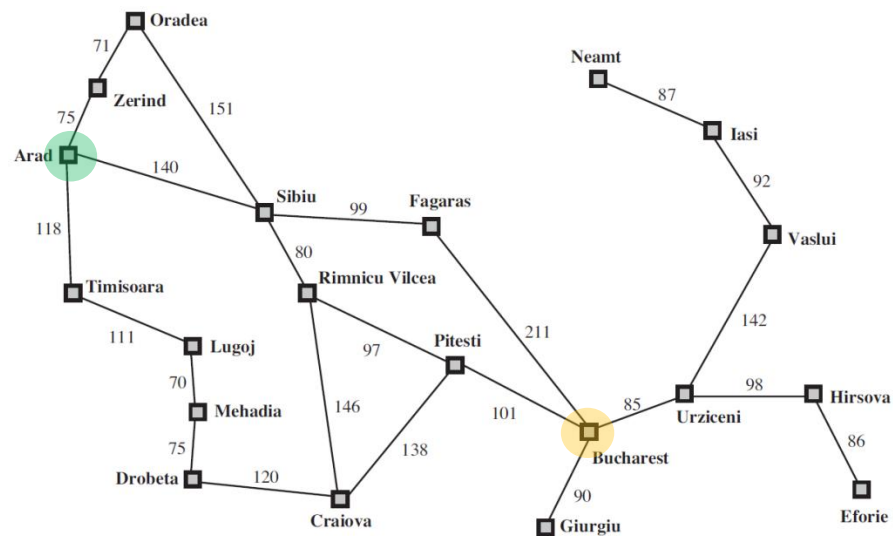
Požrešno iskanje

- angl. *greedy best-first search*
- vedno razvijemo najbolj obetavno vozlišče **glede na hevristično oceno**
- vrednotenje vozlišča: $f(n) = h(n)$
- primer: iskanje optimalne poti z upoštevanjem najkrajše zračne razdalje
- zračne razdalje do cilja (Bukarešta) lahko uporabimo kot hevristične ocene:

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



Požrešno iskanje: primer



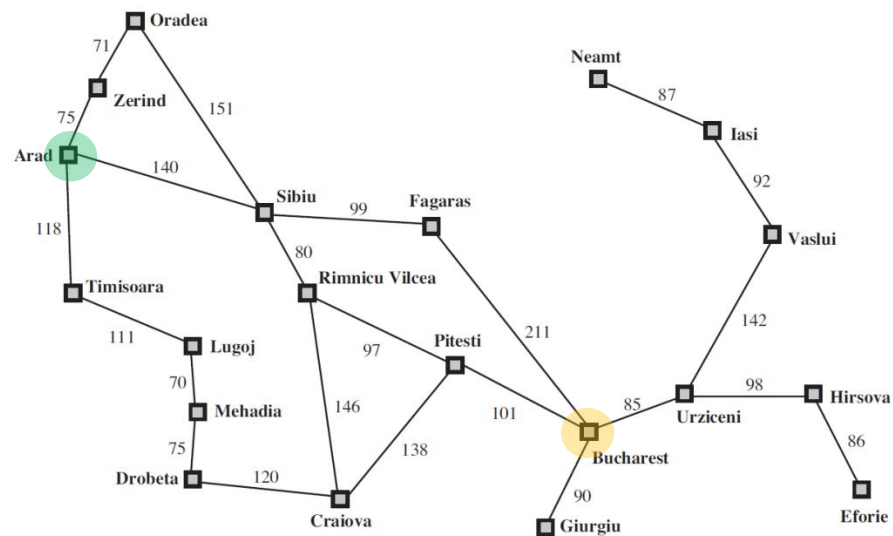
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244

Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

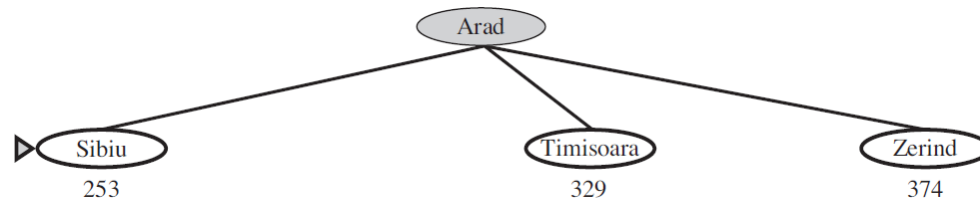


hevristična
ocena

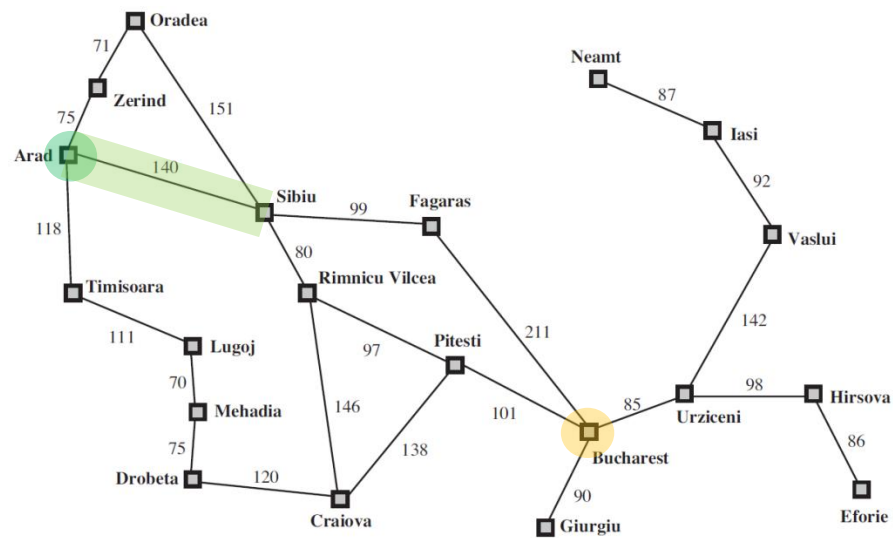
Požrešno iskanje: primer



Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

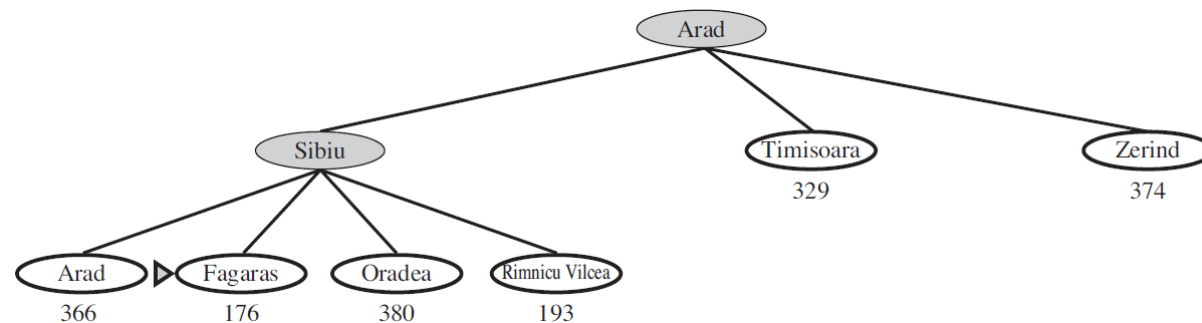


Požrešno iskanje: primer

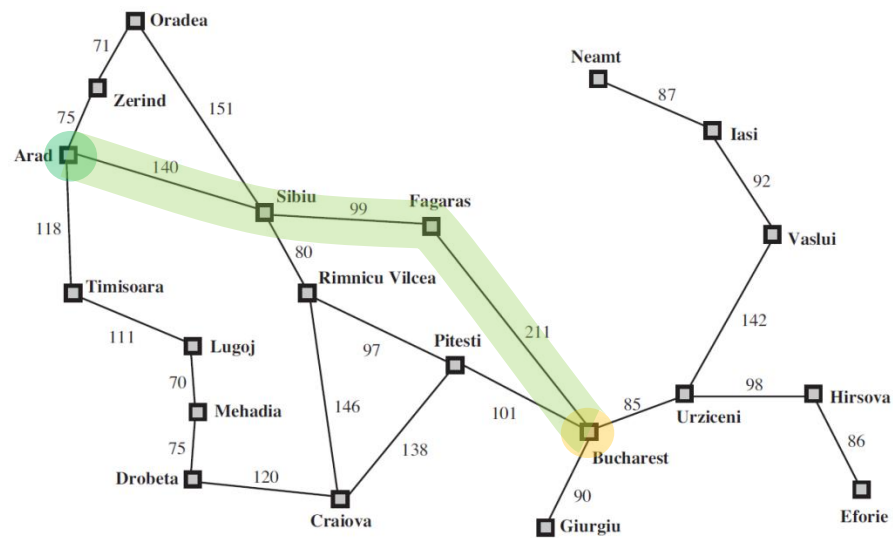


Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244

Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

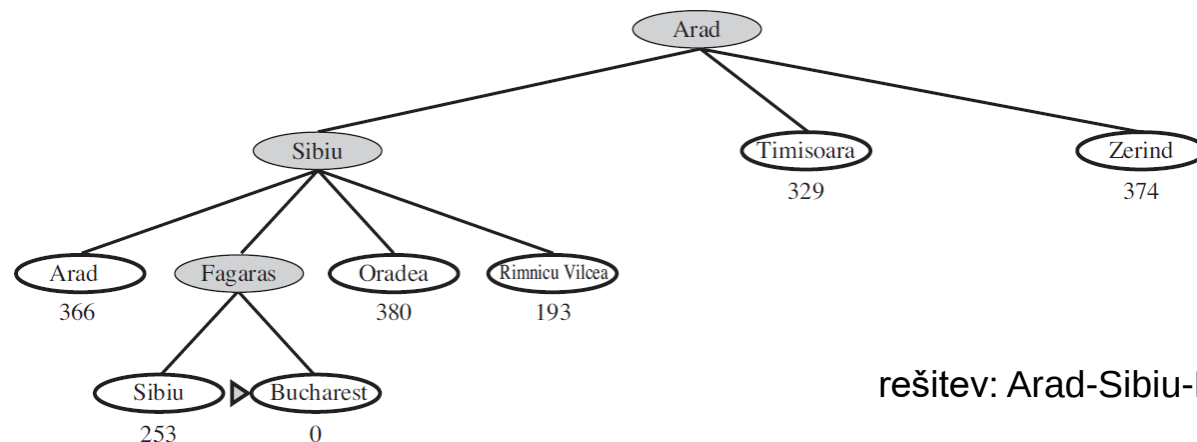


Požrešno iskanje: primer



Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244

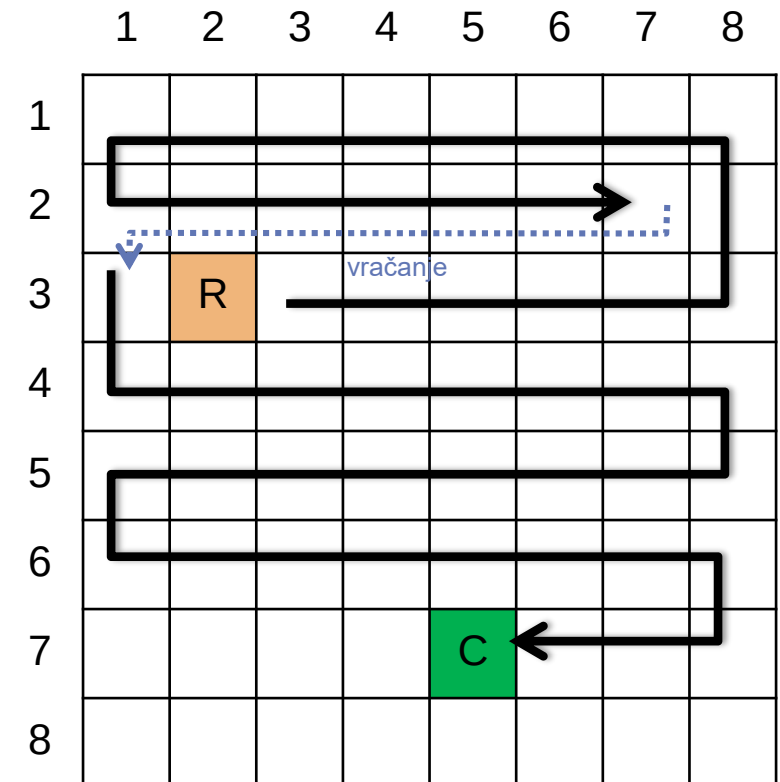
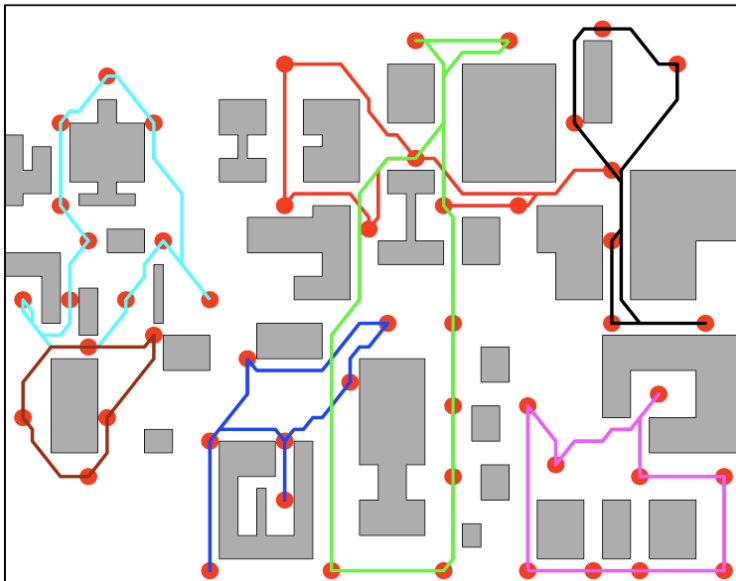
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



rešitev: Arad-Sibiu-Fagaras-Bucharest, cena=450

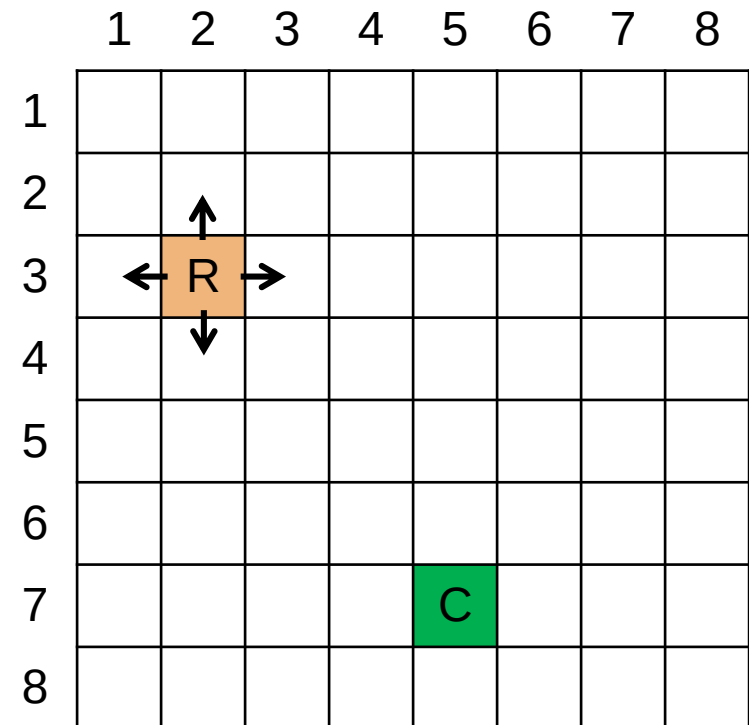
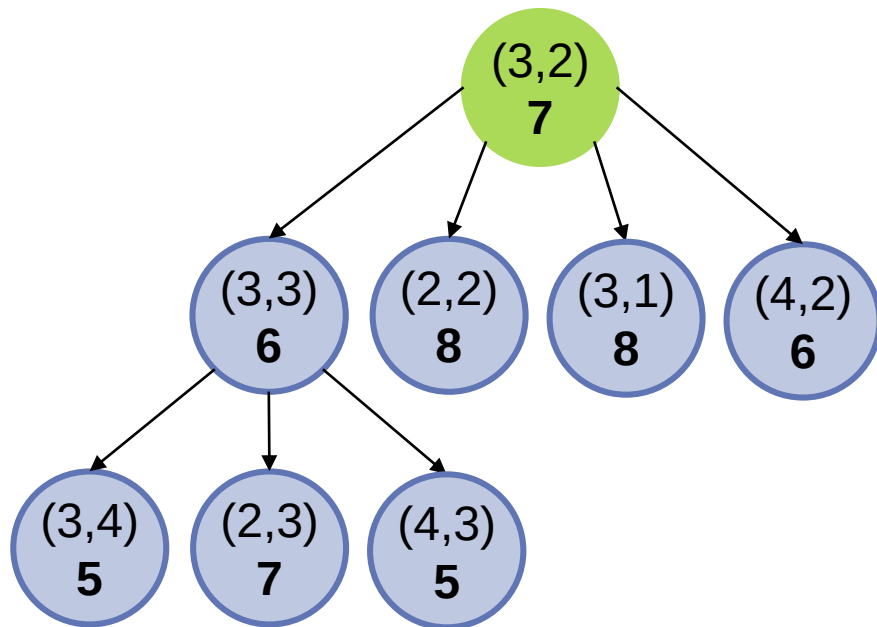
Požrešno iskanje: primer

- primer 2: robotovo iskanje ciljne lokacije (Bratko, OUI, 2016/17)
- R možni premiki: $1 \rightarrow$, $2 \uparrow$, $3 \leftarrow$, $4 \downarrow$
- dolžina rešitve pri **preiskovanju v globino** je: 45
- optimalna dolžina rešitve: 7



Požrešno iskanje: primer

- ideja: uporabimo hevristiko, ki ocenjuje obetavnost koordinate za doseganje do cilja - npr. manhattanska razdalja
- izberemo in razvijemo vozlišče glede na najmanjšo ocenjeno oceno (hevristiko)

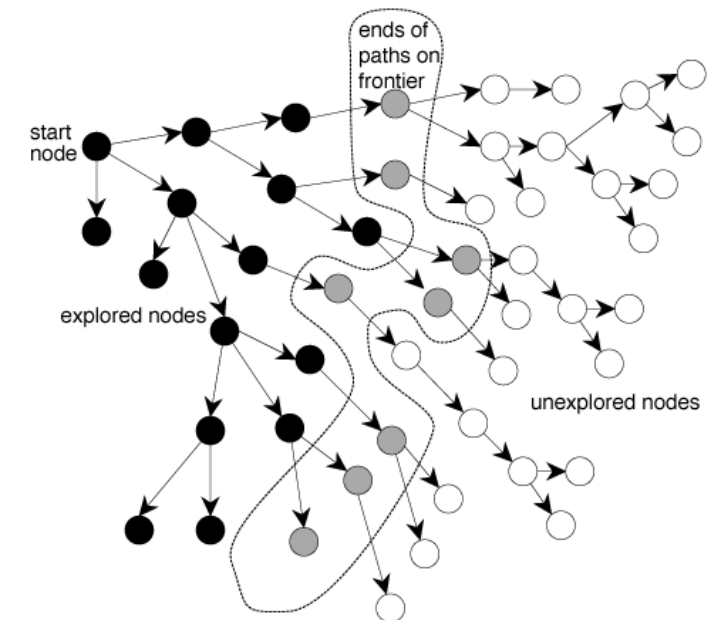
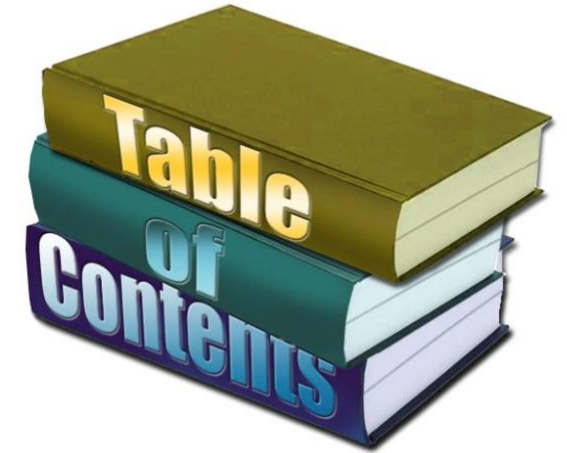


Učinkovitost požrešnega iskanja

- **POPOLNOST** (angl. *completeness*):
 - Ne.
 - Možnost ciklanja v lokalnih delih grafa (primer: pri iskanju poti Iasi->Fagaras se lahko zaciklamo med Iasi $\Leftrightarrow$ Neamt).
- **OPTIMALNOST** (angl. *optimality*):
 - Ne.
 - Ali obstaja v našem primeru iskanja najkrajše poti bolj optimalna pot do cilja?
- **ČASOVNA in PROSTORSKA ZAHTEVNOST:**
 - $O(b^m)$, kjer je m največja globina drevesa
 - vsa vozlišča moramo hraniti v spominu, ker so kandidati za razvijanje nadaljnje poti
 - pomembnost ustrezne hevristične ocene (!)

Pregled

- preiskovanje prostora stanj
 - definicija in cilji področja
 - primeri umetnih in realnih problemov
 - neinformirani preiskovalni algoritmi
 - iskanje v širino
 - iskanje v globino
 - iterativno poglobljanje
 - dvosmerno iskanje
 - cenovno – optimalno iskanje
 - informirani preiskovalni algoritmi
 - hevristično preiskovanje
 - požrešno preiskovanje
 - A*
 - IDA*
 - kakovost hevrističnih funkcij



Algoritem A*

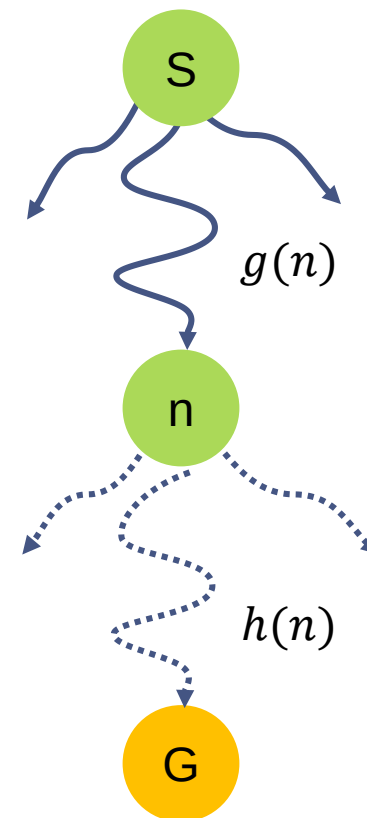
- ideja: izboljšajmo hevristično funkcijo, ker so od nje očitno odvisni uspešnost iskanja in poraba časa/prostora
- vozlišča vrednotimo glede na ceno najboljše poti skozi vozlišče n :

$$f(n) = g(n) + h(n)$$

$g(n)$ – cena poti do n (znano)

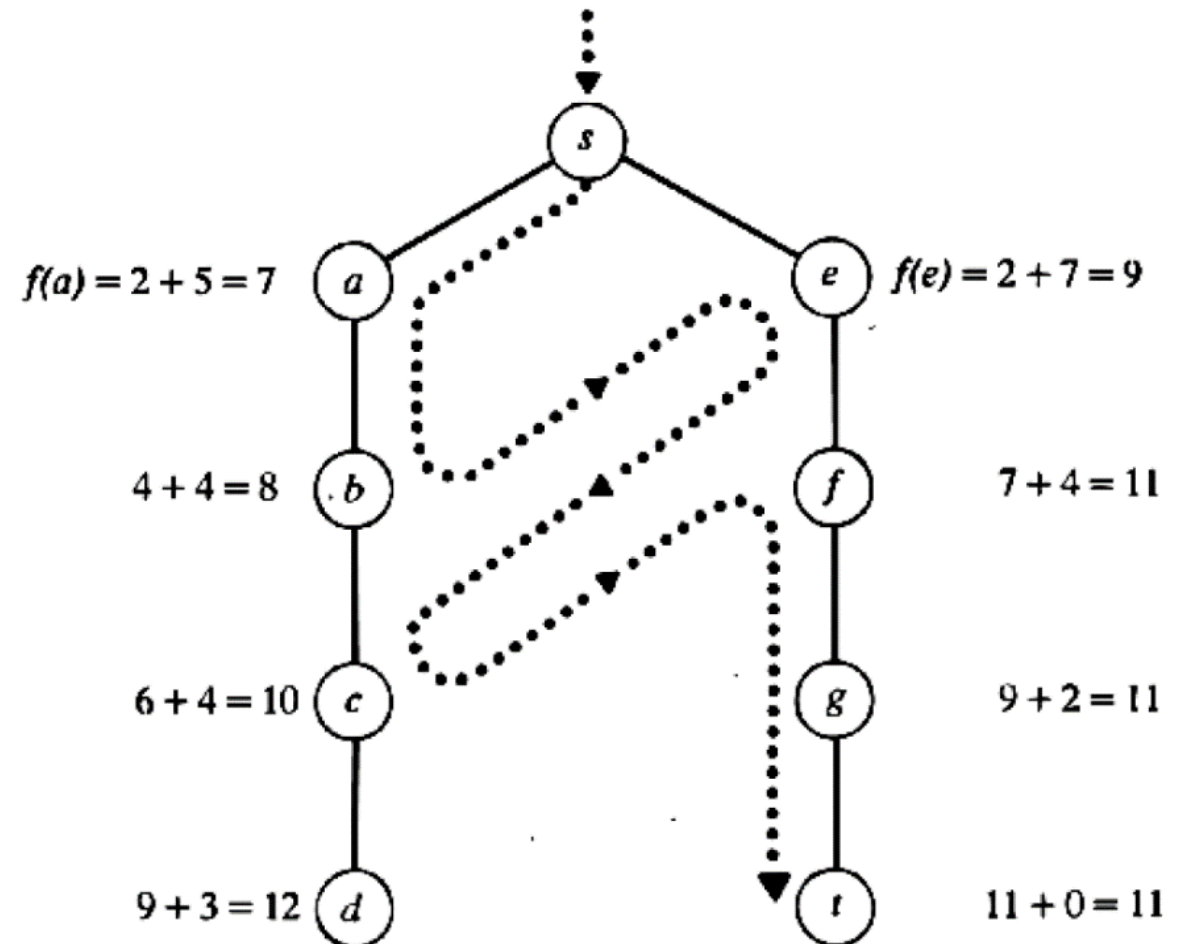
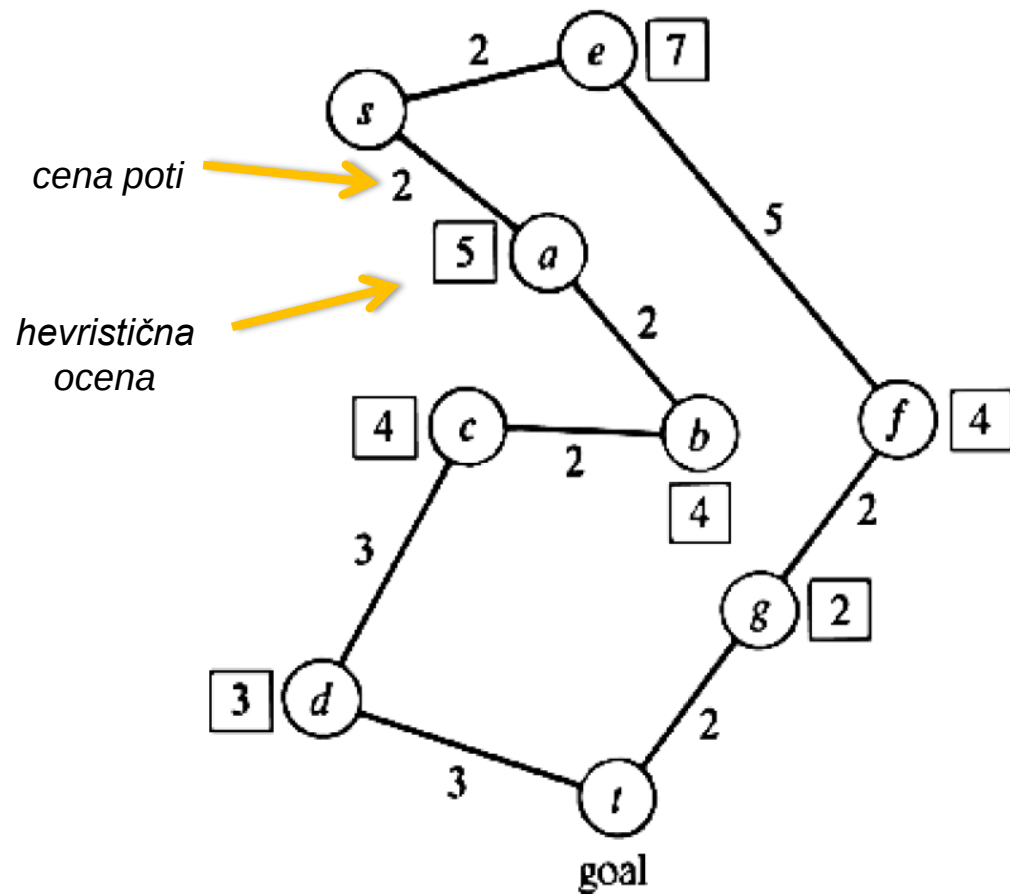
$h(n)$ – cena od n do najbližjega cilja (ocena)

- vozlišča v fronti hranimo v **prioritetni vrsti**, ki je urejena naraščajoče glede na funkcijo $f(n)$
- pri preiskovanju lahko **ponovno generiramo vozlišče n** , ki je že med razvitimi vozlišči (n')
 - če je $g(n') < g(n)$, smo našli boljšo pot do n , vozlišče dodamo v fronto
 - če je $g(n') \geq g(n)$, lahko ponovno generirano vozlišče n ignoriramo



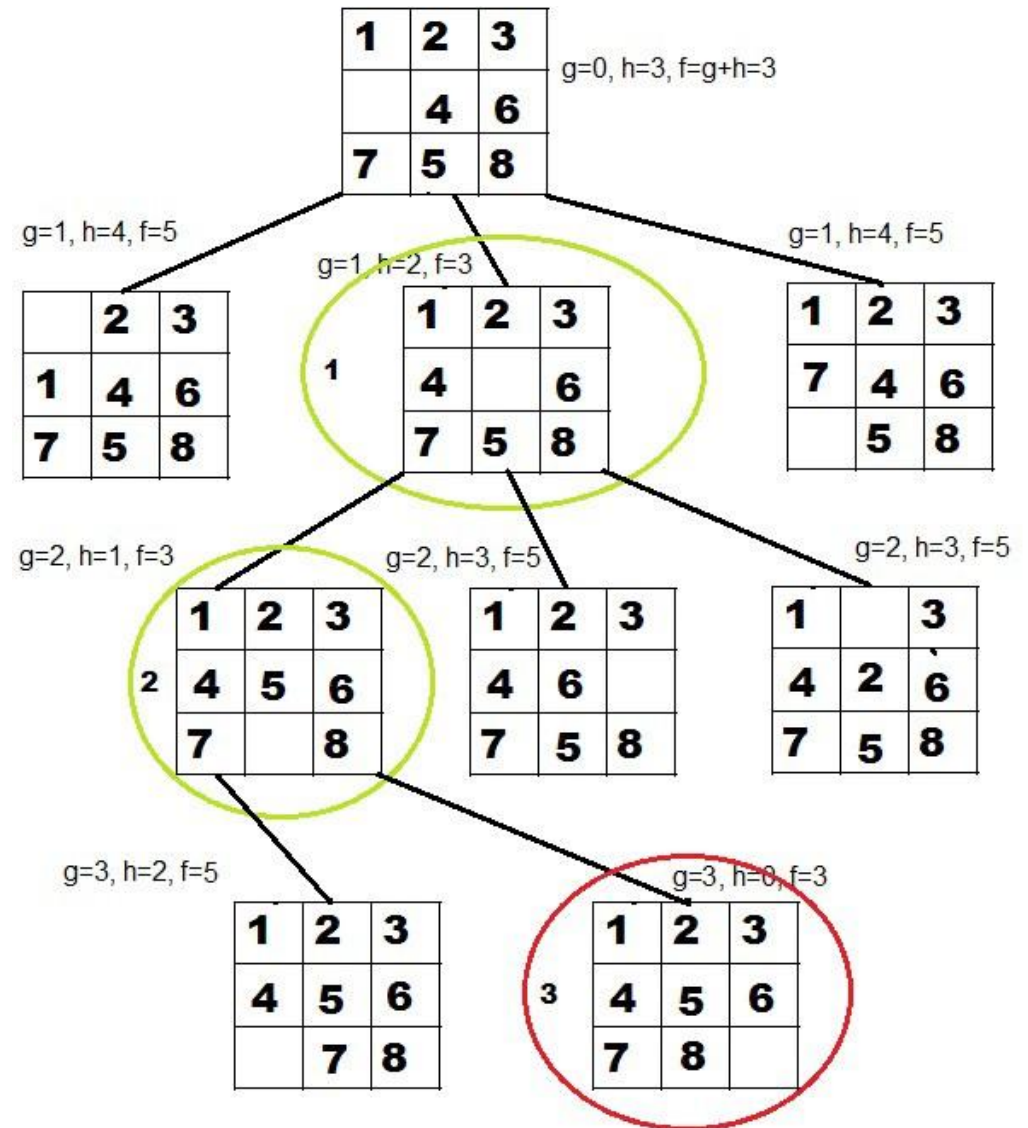
Algoritem A*: primer

- primer 1: preiskovanje manjšega grafa



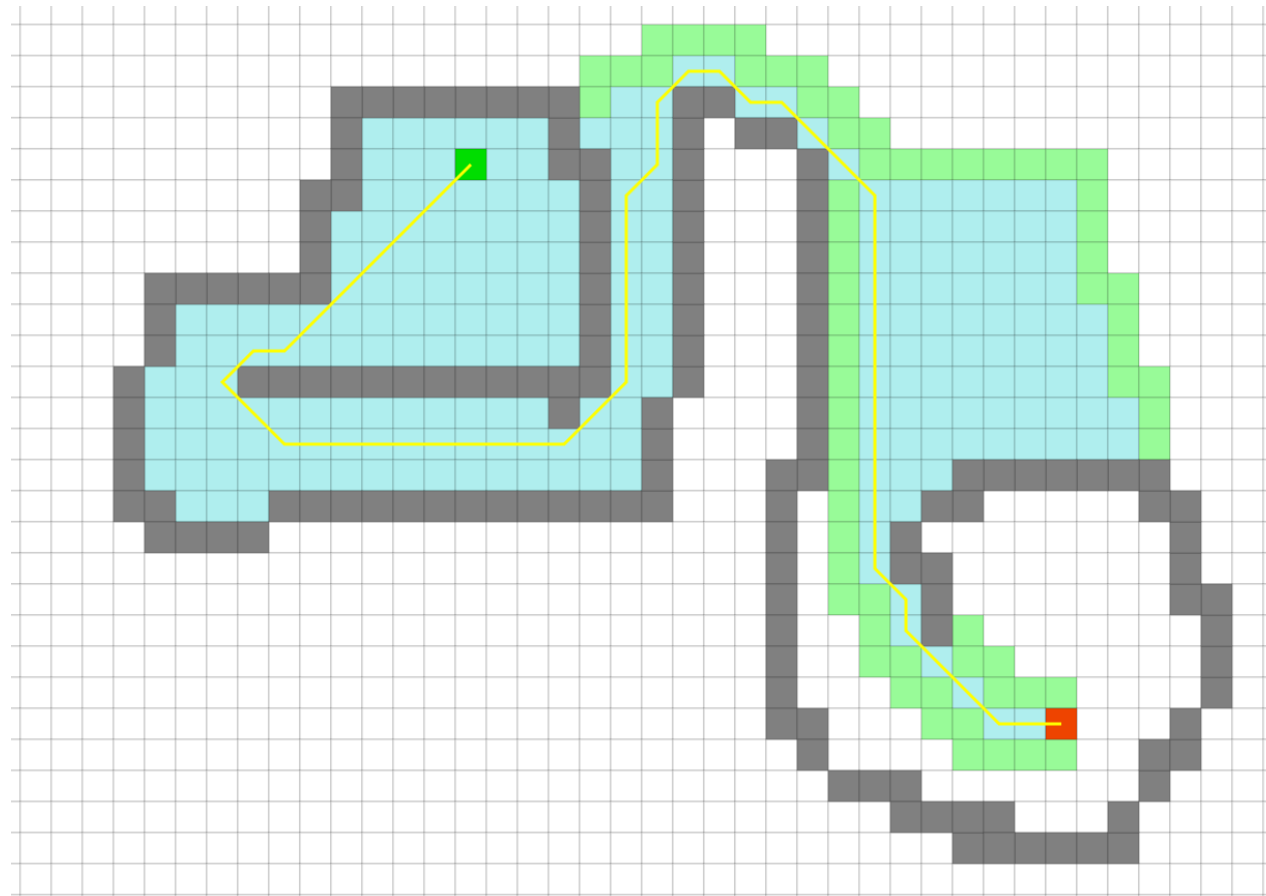
Algoritem A*: primer

- primer 2: igra 8 ploščic
- hevrstika: koliko ploščic ni na pravem mestu

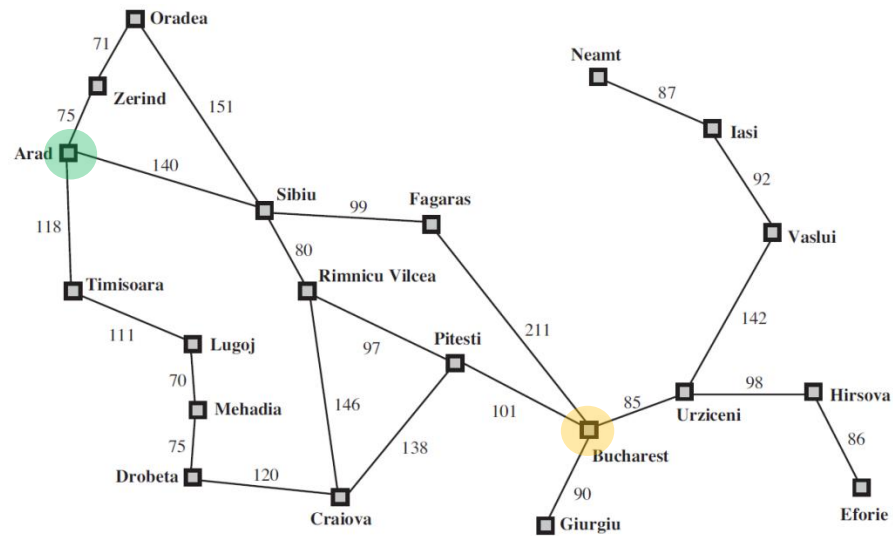


Algoritme A*: primer

- primer 3: <https://qiao.github.io/PathFinding.js/visual/>



Algoritem A*: primer



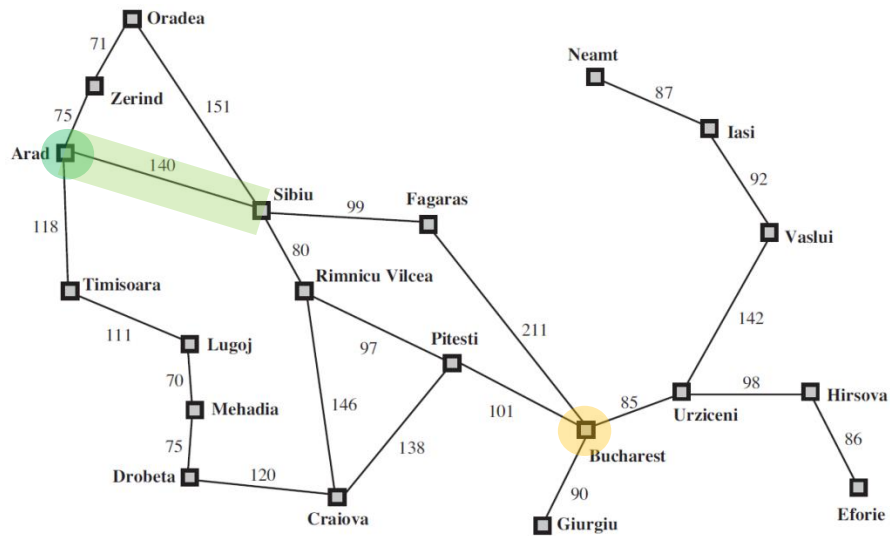
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244

Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

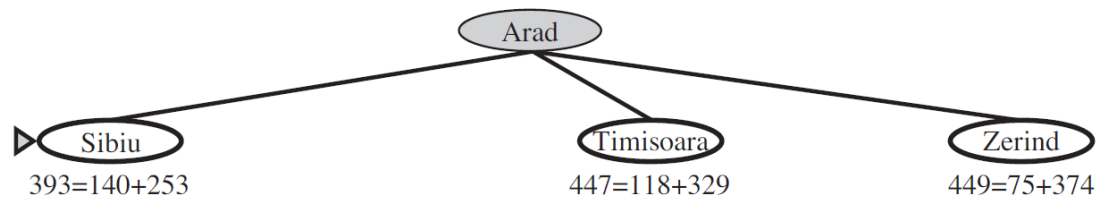
► Arad

$$366=0+366$$

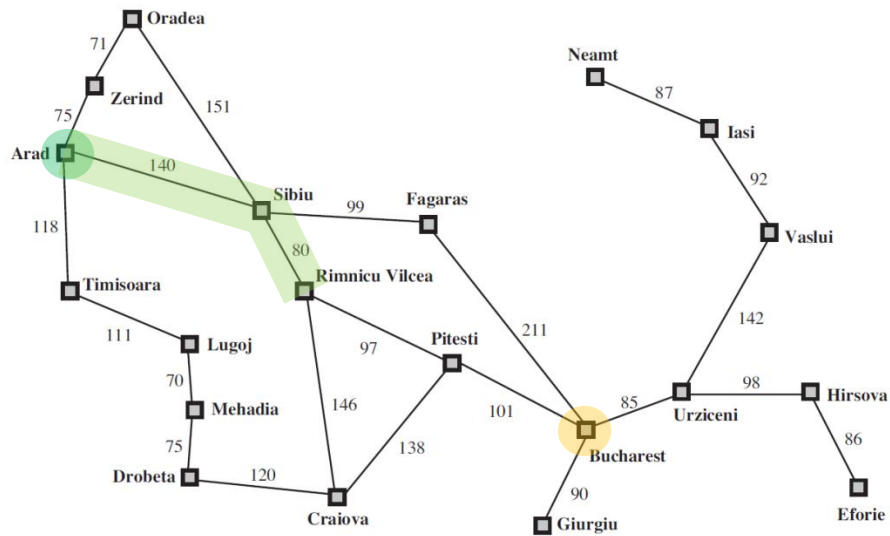
Algoritem A*: primer



Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

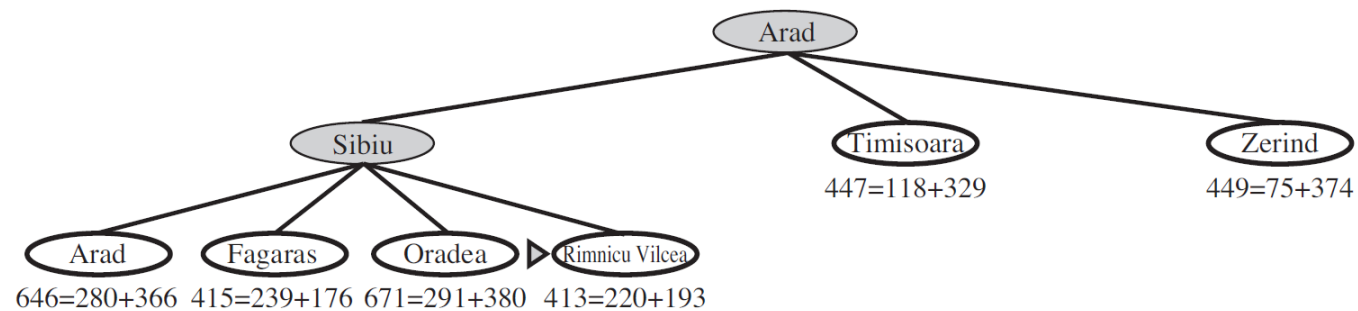


Algoritem A*: primer

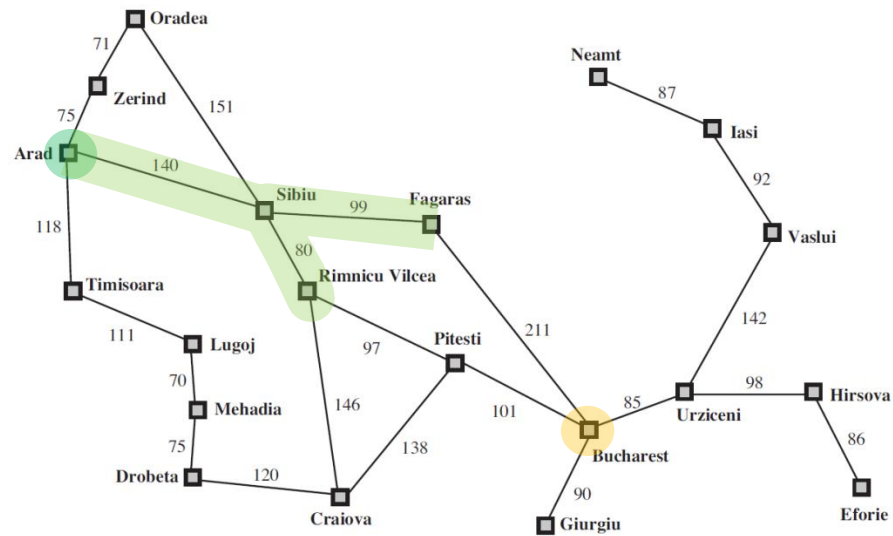


Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244

Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

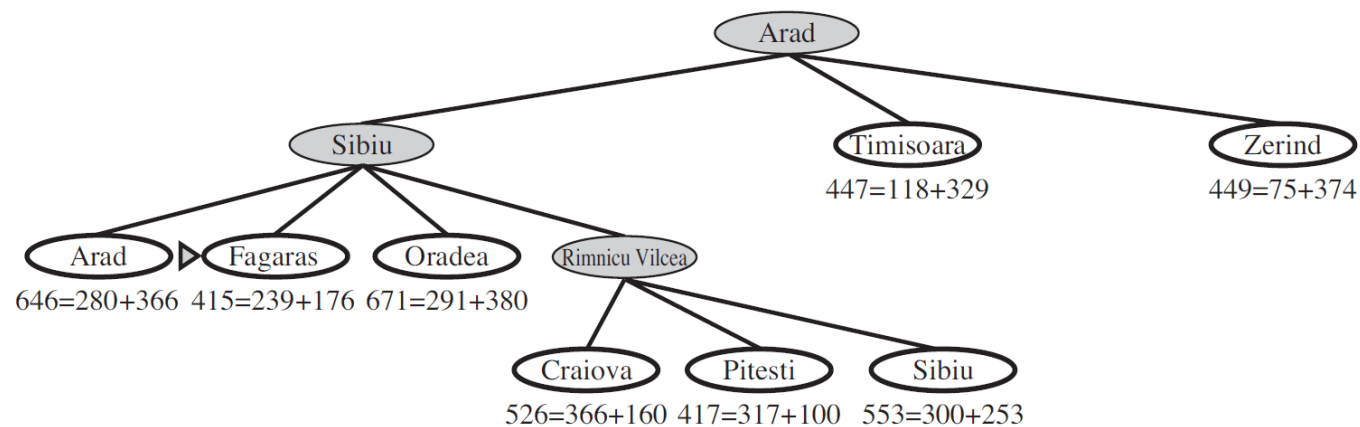


Algoritem A*: primer

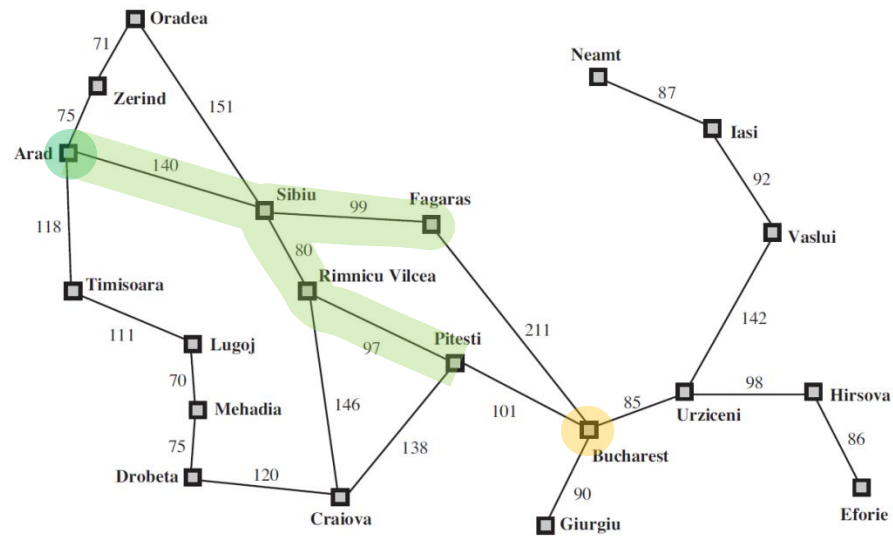


Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244

Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

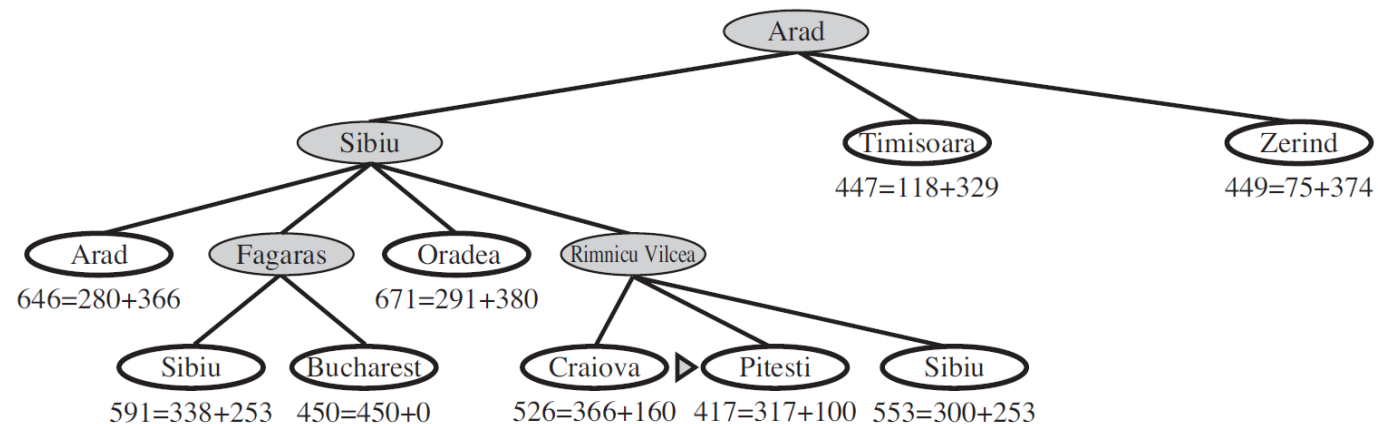


Algoritem A*: primer

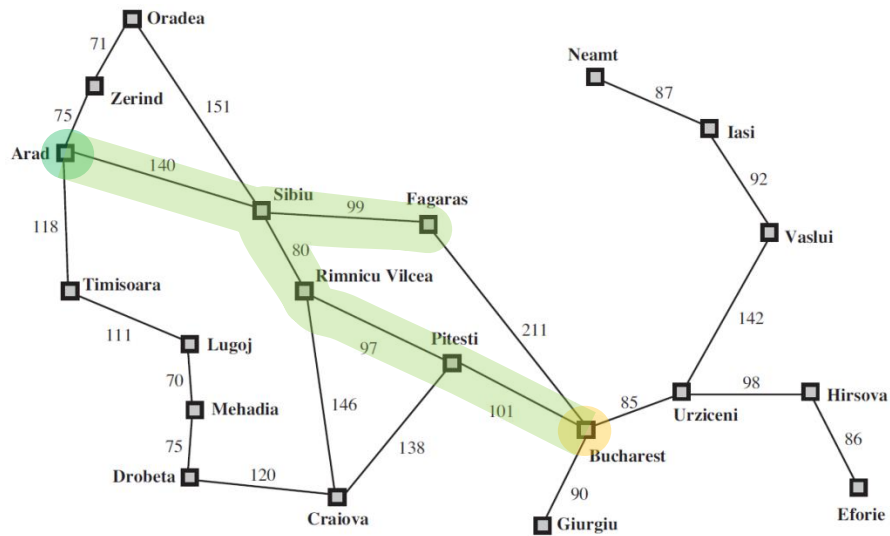


Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244

Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

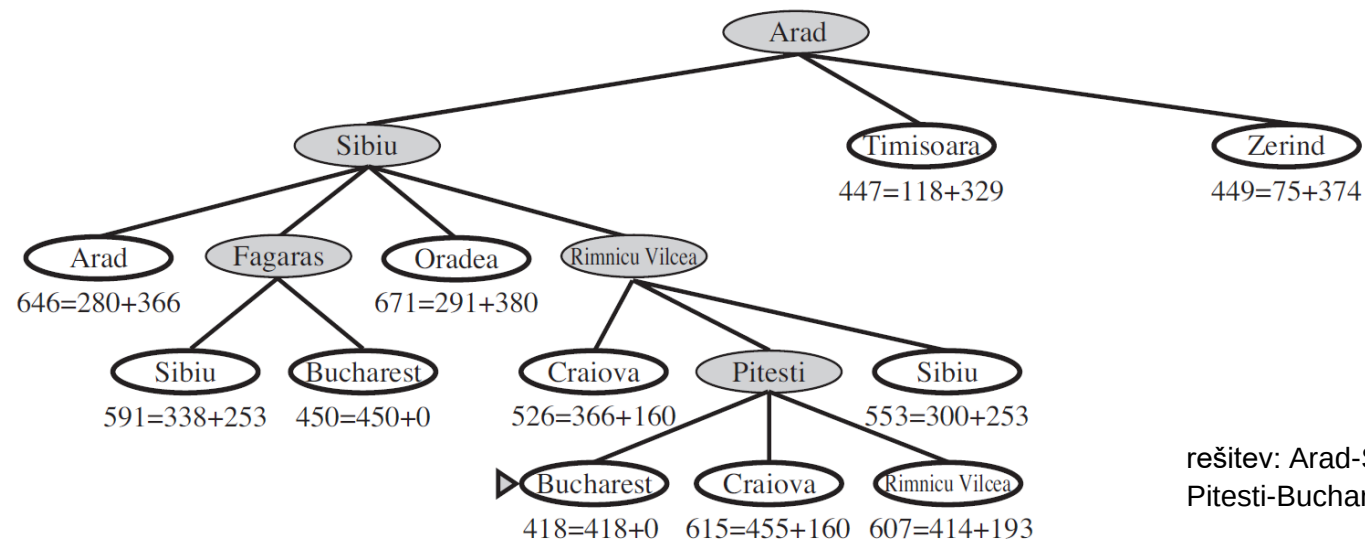


Algoritem A*: primer



Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244

Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



rešitev: Arad-Sibiu-Rimnicu Vilcea-Pitesti-Bucharest, cena=418

Popolnost in optimalnost A*

- algoritem A* je **popoln** in **optimalen**, če ustreza pogoju **dopustnosti** (angl. *admissibility*)
- za hevristiko $h(n)$ pravimo, da je **dopustna**, če nikoli **ne precenjuje cene do cilja**
 - formalno: hevristika $h(n)$ je dopustna, če za vsako vozlišče n velja $h(n) \leq h^*(n)$, kjer je $h^*(n)$ dejanska cena optimalne poti do cilja za vozlišče n
 - zgornje pomeni, da je hevristika $h(n)$ "**optimistična**" (= predvideva, da je do cilja manj, kot dejansko je)
 - posledično tudi $f(n)$ ne precenjuje cene do cilja, saj je $g(n)$ znan, velja pa $f(n) = g(n) + h(n)$
- ali je lahko $h(n) = 0$
(to je tudi optimistična cenilka)?
Da, vendar... .. ?
- idealno velja $h(n) = h^*(n)$

A* QUESTION

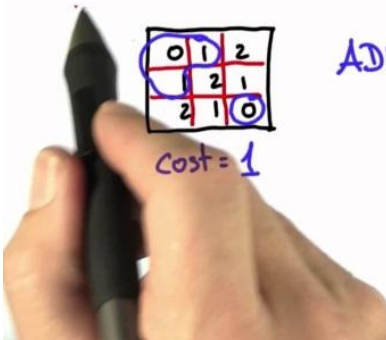
ADMISSIBLE HEURISTIC $h(x) \leq \text{cost-to-goal}$

0	1	2
1	2	1
2	1	0

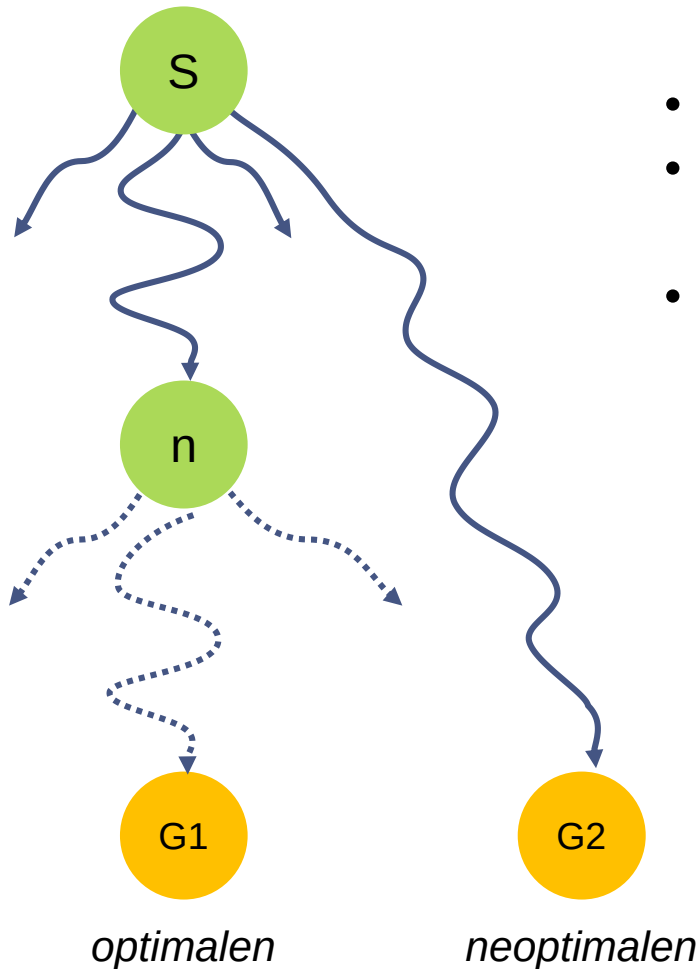
cost = 1

ADMISSIBLE ?

~~YES~~ NO



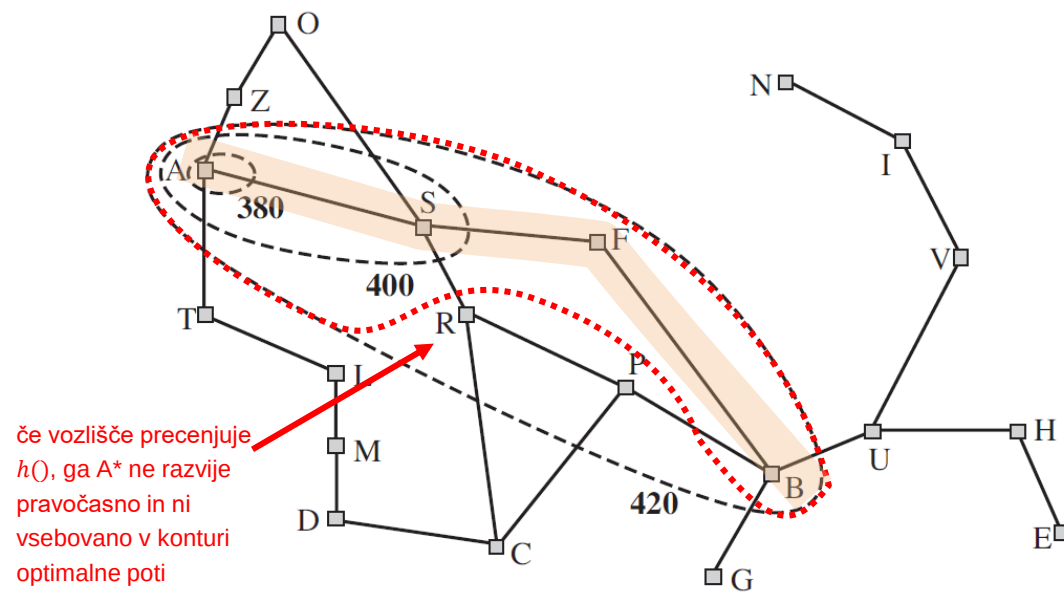
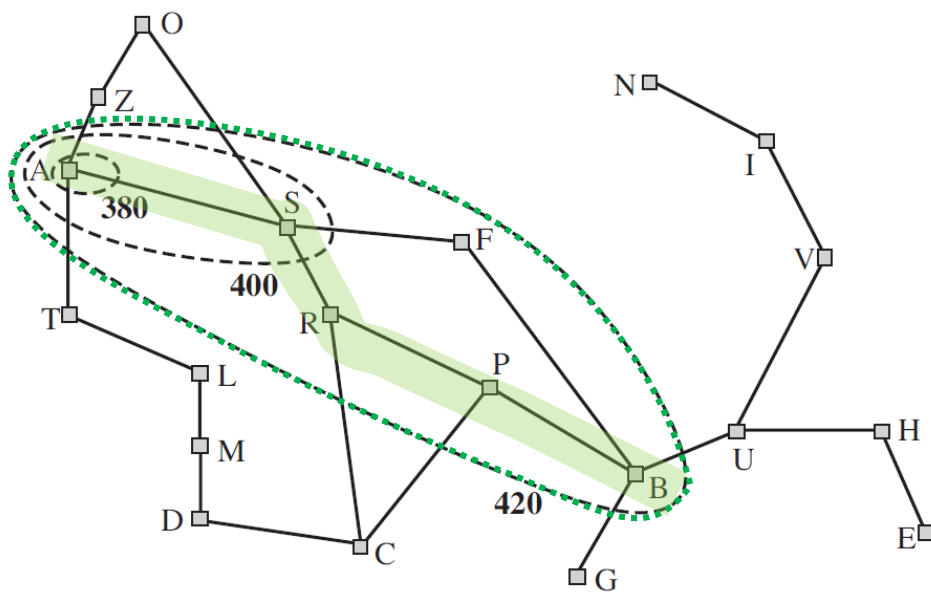
Skica dokaza optimalnosti A*



- denimo, da je $G1$ je optimalen cilj, $G2$ neoptimalen ($g(G1) < g(G2)$)
- denimo, da je $h(n)$ dopustna cenilka (za vsak n velja: $h(n) \leq h^*(n)$)
- dokaz s protislovjem:
 - predpostavimo, da A* najde drugi neoptimalen cilj ($G2$)
 - če $G2$ ni optimalen, mora zanj veljati $f(G2) > f(G1)$
 - ker je $h(n)$ dopustna cenilka, velja za vsa vozlišča n na optimalni poti (do $G1$): $f(n) \leq f(G1)$
 - iz zgornjih dveh alinej velja $f(n) \leq f(G1) < f(G2)$
 - ker A* razvija vozlišča v vrstnem redu glede na manjši $f(n)$, ne more najti cilja $G2$, saj ima ta višji $f(G2)$ $\Rightarrow$ protislovje

Skica dokaza optimalnosti A*

- še drugačen premislek o optimalnosti
- algoritem A* razvija vozlišča glede na naraščajočo oceno vozlišč $f(n)$. Pri tem povečuje "raziskanost" prostora v obliki **reliefnih kontur** (vsaka kontura predstavlja večjo vrednost $f(n)$)
- če heuristika ne bi bila dopustna (in bi v nekem vozlišču precenjevala ceno do cilja), to vozlišče ne bi bilo vsebovano v konturi, ki predstavlja vrednost optimalne poti do cilja
- primer prikazuje:
 - levo: optimalna pot
 - desno: A* bi zaobšel vozlišče R, ki je na optimalni poti, če bi imel preveliko vrednost $f(n)$



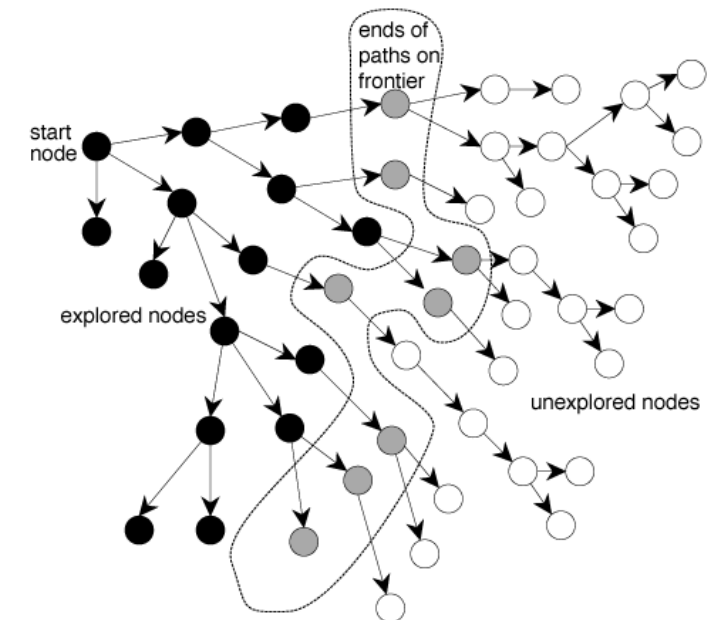
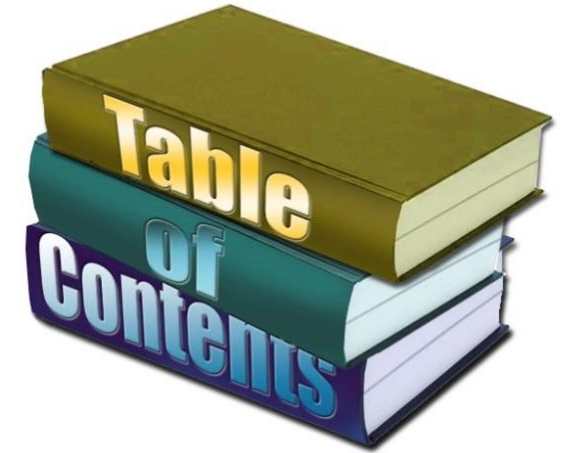
Učinkovitost algoritma A*

- **POPOLNOST in OPTIMALNOST:**
 - Da, če je heuristika **dopustna**.
- **ČASOVNA ZAHTEVNOST:**
 - odvisni sta od kakovosti heuristike $h(n)$
(boljša heuristika – manjša poraba časa in prostora)
 - definirajmo:
 - h^* naj bo dejanska cena do optimalne rešitve
 - relativna napaka heuristike $\epsilon = (h^* - h)/h^*$
 - zahtevnost je eksponentna glede na funkcijo relativne napake in globino rešitve: $O(b^{f(\epsilon) \cdot d})$
- **PROSTORSKA ZAHTEVNOST:**
 - večji problem kot časovna zahtevnost, ker mora A* hraniti vsa vozlišča v spominu
 - nepraktično za velike probleme
 - boljše alternative glede porabe prostora: algoritem IDA* (*iterative deepening A**), RBFS (*recursive best-first search*), MA* (*memory-bounded A**), SMA* (*simplified A**), LRTA* (*learning real-time A**)



Pregled

- preiskovanje prostora stanj
 - definicija in cilji področja
 - primeri umetnih in realnih problemov
 - neinformirani preiskovalni algoritmi
 - iskanje v širino
 - iskanje v globino
 - iterativno poglobljanje
 - dvosmerno iskanje
 - cenovno – optimalno iskanje
- informirani preiskovalni algoritmi
 - hevristično preiskovanje
 - požrešno preiskovanje
 - A^*
 - IDA*
 - kakovost hevrističnih funkcij



Algoritem IDA*

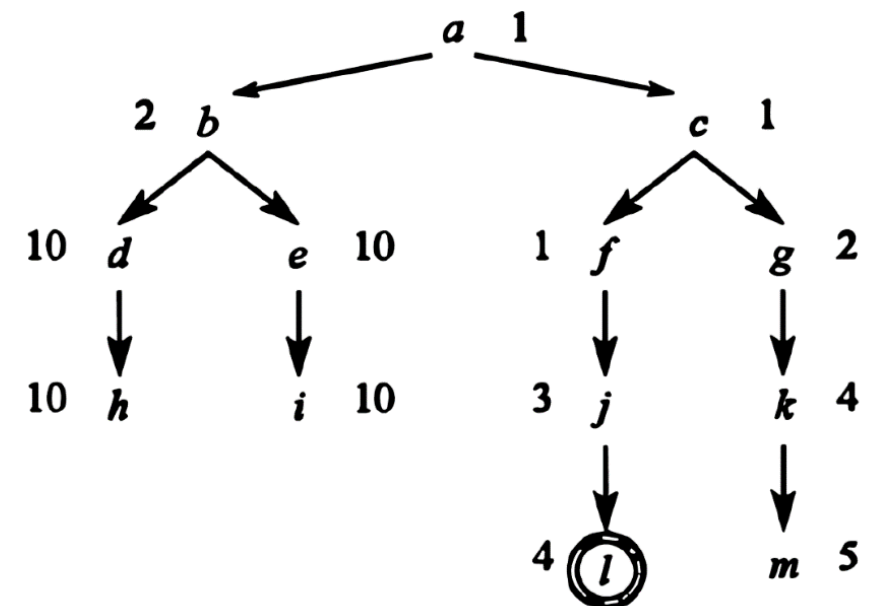
- iterative-deepening A*
- deluje analogno kot **iterativno poglobljanje** (izvaja **preiskovanje v globino** z različnimi mejami), vendar za mejo ne uporabljamo globine, temveč vrednost funkcije $f(n)$
 - za mejo na začetku izberemo vrednost $f(n)$ začetnega vozlišča
 - na vsaki iteraciji razvijemo vsa vozlišča z $f(n) \leq$ mejni vrednosti
 - za naslednjo iteracijo izberemo mejo, ki je najmanjši $f(n)$ še nerazvitih vozlišč

```
procedure ida_star(root)
  bound := h(root)
  path := [root]
  loop
    t := search(path, 0, bound)
    if t = FOUND then return (path, bound)
    if t = ∞ then return NOT_FOUND
    bound := t
  end loop
end procedure
```

```
function search(path, g, bound)
  node := path.last
  f := g + h(node)
  if f > bound then return f
  if is_goal(node) then return FOUND
  min := ∞
  for succ in successors(node) do
    if succ not in path then
      path.push(succ)
      t := search(path, g + cost(node, succ), bound)
      if t = FOUND then return FOUND
      if t < min then min := t
      path.pop()
    end if
  end for
  return min
end function
```

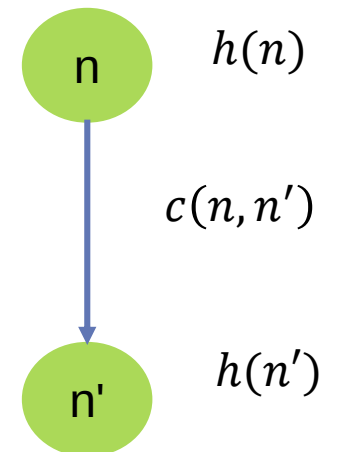
Algoritem IDA*

- primer:
 - podane so vrednost $f(n)$ ($= g(n) + h(n)$) vozlišč
 - simuliraj preiskovanje z IDA*
- generirana vozlišča
 - 1. iteracija, meja=1: a/1, b/2, c/1, f/1, j/3, g/2
 - 2. iteracija, meja=2: a/1, b/2, d/10, e/10, c/1, f/1, j/3, g/2, k/4
 - 3. iteracija, meja=3: a/1, b/2, d/10, e/10, c/1, f/1, j/3, l/4, g/2, k/4
 - 4. iteracija, meja=4: a/1, b/2, d/10, e/10, c/1, f/1, j/3, l/4



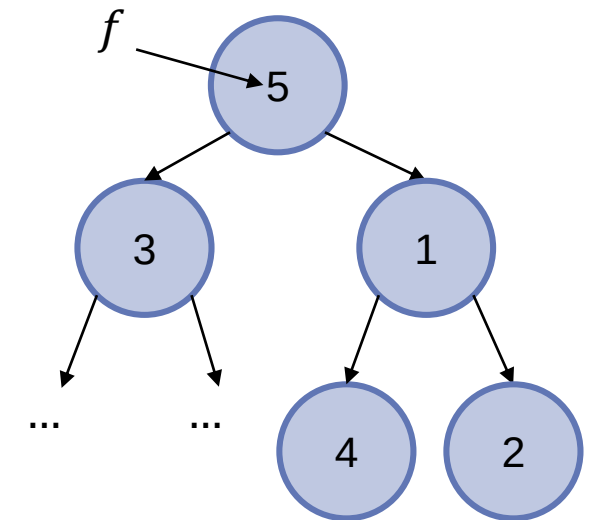
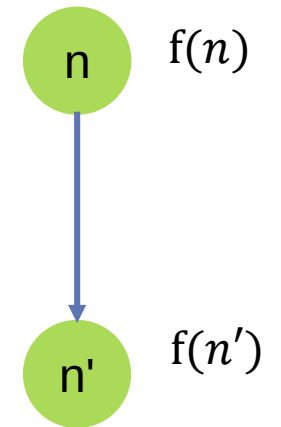
Učinkovitost algoritma IDA*

- redundanca: ponovno generiranje velikega števila vozlišč
- neučinkovit, če prevladujejo vozlišča z zelo raznolikimi vrednostmi funkcije $f(n)$
- vendar: v spominu hrani samo trenutno pot (podobno kot pri iskanju v globino) in ne vseh vozlišč kot A*
- želimo si, da je IDA* optimalen glede na: ceno najdene rešitve, porabljen prostor in porabljen čas:
 - to je možno, če IDA* razvije najmanjše potrebno število vozlišč, čemur rečemo, da jih razvija v *prioritetnem vrstnem redu*
 - IDA* razvija vozlišča v prioritetnem vrstnem redu, če je hevristična ocena $h(n)$ **monotona** ali **konsistentna** (monotone/consistent). To je res, kadar za vsaki povezani vozlišči n in n' velja (trikotniška neenakost):
$$h(n) \leq c(n, n') + h(n')$$
in $h(g) = 0$ za vsako končno vozlišče g
 - če je hevristika monotona/konsistentna, je tudi dopustna (!)



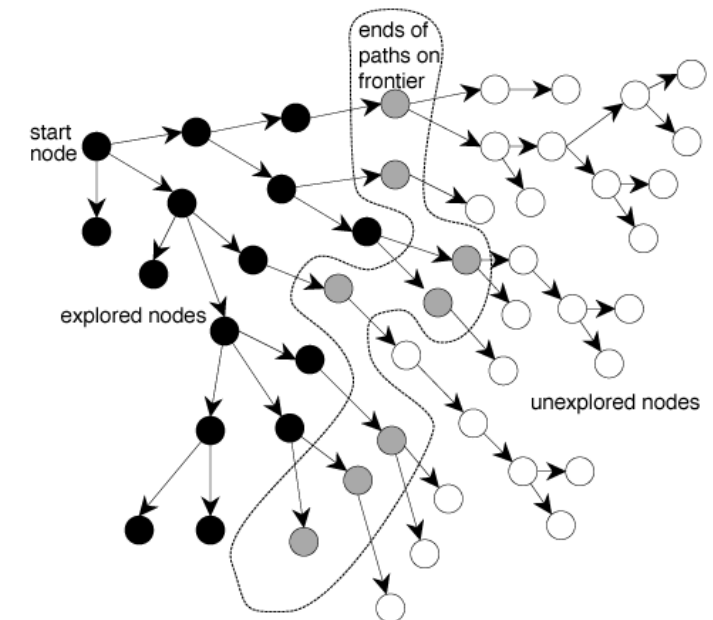
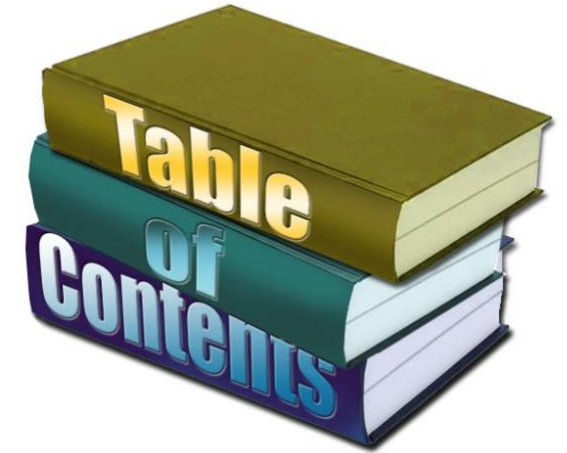
Učinkovitost algoritma IDA*

- poenostavitev: za zagotovitev razvijanja vozlišč v prioritetenem vrstnem redu ni nujno potrebno, da velja monotonost za hevristiko h , temveč **že zadošča, da je monotona cenilna funkcija $f (= g + h)$**
- cenilna funkcija f je monotona, če za vsak par povezanih vozlišč n in n' velja $f(n) \leq f(n')$
- primer nemonotone funkcije f :
 - meja za f je enaka 5, vendar pa vozlišče z $f = 3$ razvijemo pred vozlišči z $f = 1$ in $f = 2$
- premislek – izziv (objavi odgovor na forumu!)
 - Ali dopustna hevristična funkcija h zagotavlja, da je cenilna funkcija f monotona?
 - Ali za monotone cenilne funkcije f velja, da zadoščajo pogoju iz izreka o dopustnosti hevristik h ?



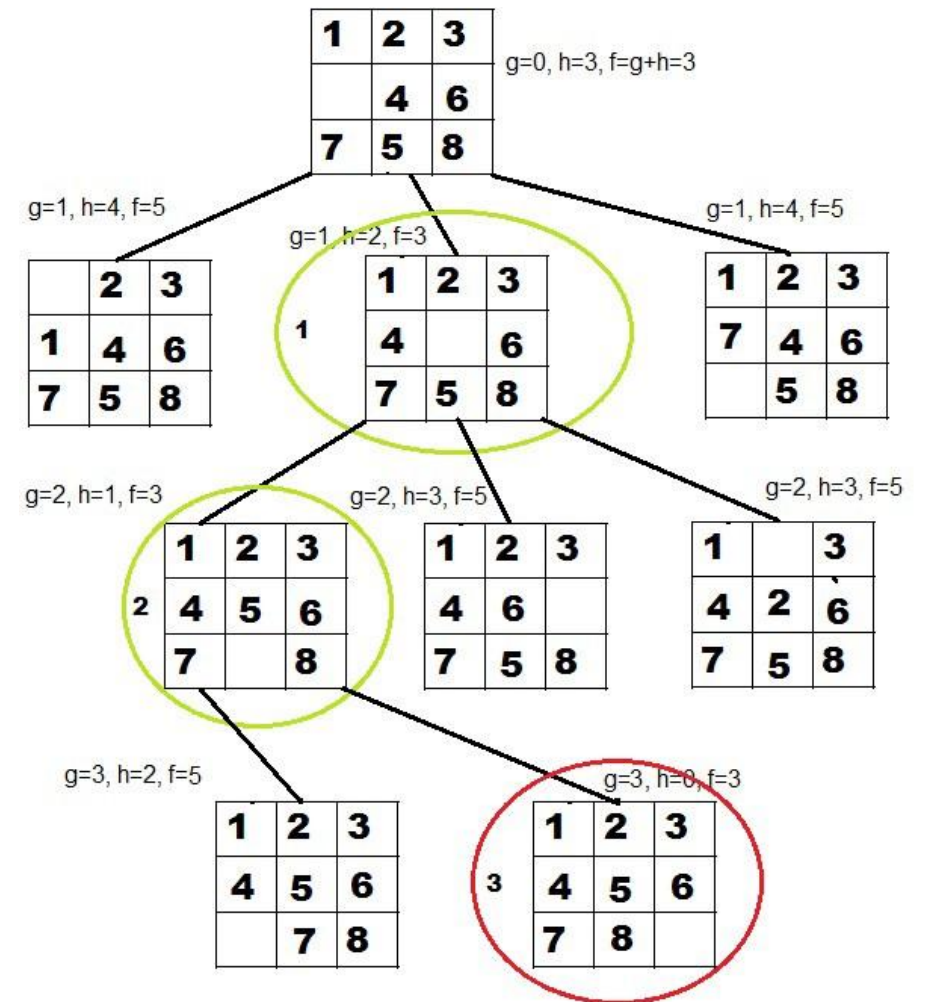
Pregled

- preiskovanje prostora stanj
 - definicija in cilji področja
 - primeri umetnih in realnih problemov
 - neinformirani preiskovalni algoritmi
 - iskanje v širino
 - iskanje v globino
 - iterativno poglobljanje
 - dvosmerno iskanje
 - cenovno – optimalno iskanje
- informirani preiskovalni algoritmi
 - hevristično preiskovanje
 - požrešno preiskovanje
 - A^*
 - IDA^*
 - kakovost hevrističnih funkcij



Kakovost hevrističnih funkcij

- primer: igra 8 ploščic
- povprečna dolžina rešitve je 22 korakov
- povprečni faktor vejanja je 3
(2 potezi možni v vogalu, 3 ob robu, 4 v sredini)
- izčrpno preiskovanje bi torej pregledalo prostor 3^{22} stanj (za 3x3); na srečo je dosegljivih le približno $9!/2 = 181,440$ stanj
- hevristična funkcija lahko učinkovito zmanjša prostorsko in časovno ceno iskanja
- pri iskanju ustrezne hevristike si pomagamo s poznavanjem problema



Kakovost hevrističnih funkcij

- primeri hevrističnih funkcij:
 - h_1 – število ploščic, ki niso na pravem mestu (za primer na desni: $h_1 = 8$)
 - h_2 – vsota manhattanskih razdalj ploščic do pravega mesta (za primer na desni: $h_2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$)
- kakovost h lahko ocenimo:
 - s številom generiranih vozlišč
 - z efektivnim faktorjem vejanja (koliko vozlišč N je algoritem generiral, da je na globini d našel rešitev)

7	2	4
5		6
8	3	1

Globina	število generiranih vozlišč			efektivni faktor vejanja		
	IDS	$A^*(h_1)$	$A^*(h_2)$	IDS	$A^*(h_1)$	$A^*(h_2)$
2	10	6	3	2,45	1,79	1,79
4	112	13	12	2,87	1,48	1,45
6	680	20	18	2,73	1,34	1,30
8	6384	39	25	2,80	1,33	1,24
10	47127	93	39	2,79	1,38	1,22
12	3644035	227	73	2,78	1,42	1,24
14	?	539	113	?	1,44	1,23
16	?	1301	211	?	1,45	1,25
18	?	3056	363	?	1,46	1,26
20	?	7276	676	?	1,47	1,27
22	?	18094	1219	?	1,48	1,28
24	?	39135	1641	?	1,48	1,26

Kako do idej za hevristike?

- želimo imeti dopustne hevristike:
 - s čim višjimi vrednostmi
 - s sprejemljivo ceno (časom) izračuna
- v prejšnjem primeru je h_2 boljša od h_1 (ker $h_2(n) \geq h_1(n)$ za vsak n , pravimo, da h_2 **dominira** h_1)
- pridobivanje hevristik:
 - iz poenostavljenega (relaksiranega) problema:
 - "ploščico lahko prestavimo na poljubno (tudi neprazno) polje"
 - "ploščico lahko prestavimo na poljubno (tudi nesosednje) prazno polje"
 - "ploščico lahko prestavimo na sosednje (tudi neprazno) polje"
 - z vzorci podproblemov (osredotočimo se npr. samo na iskanje rešitve za del problema)
 - z izkušnjami in uteževanjem kriterijev (npr. oddaljenost od cilja, število sosednjih ploščic, ki ne mejijo na ciljno mesto ipd.)

*	2	4
*		*
*	3	1

Start State

	1	2
3	4	*
*	*	*

Goal State



**Lokalno preiskovanje,
posebna preiskovanja, igranje iger**